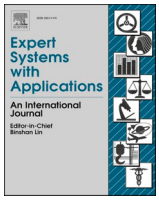
Expert Systems With Applications 264 (2025) 125861

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



LLM-TSFD: An industrial time series human-in-the-loop fault diagnosis method based on a large language model

Qi Zhang [a], Chao Xu [a], Jie Li [a], Yicheng Sun [a], Jinsong Bao [a,*], Dan Zhang [a,b]

[a] College of Mechanical Engineering, Donghua University, Shanghai 201602, China
[b] Department of Mechanical Engineering, The Hong Kong Polytechnic University, Hong Kong, China

ARTICLE INFO

Keywords:
Time series
Fault diagnosis 2.0
Task-driven
Large language model
Human-in-the-loop

ABSTRACT

Industrial time series data provides real-time information about the operational status of equipment and helps identify anomalies. Data-driven and knowledge-guided methods have become predominant in this field. However, these methods depend on industrial domain knowledge and high-quality industrial data which can lead to issues such as unclear diagnostic results and lengthy development cycles. This paper introduces a novel human-in-the-loop task-driven approach to reduce reliance on manually annotated data and improve the interpretability of diagnostic outcomes. This approach utilises a large language model for fault detection, fostering process autonomy and enhancing human–machine collaboration. Furthermore, this paper explores four key roles of the large language model: managing the data pipeline, correcting causality, controlling model management, and making decisions about diagnostic results. Additionally, it presents a prompt structure designed for fault diagnosis of time series data, enabling the large language model to realize task-driven. Finally, the paper validates the proposed framework through a case study in the context of steel metallurgy.

1. Instruction

With the development of Industry 5.0, the stable operation of machinery and equipment is important to ensure safe production. Effective monitoring, prediction, detection, and diagnosis of machine equipment not only guarantee industrial development but also serve as a driving force. The task of time series anomaly detection in industrial scenarios is complex, requiring the transition from manual to automated approaches throughout the development process. In industrial applications, time-series-based fault diagnosis tasks are complex. Operators need to observe and predict changes in data trends, issue fault warnings, and analyze causal relationships between trends. In this process, the task of fault needs to evolve from manual processes towards automation. Particularly in the steel metallurgy field, automating complex task flows and massive sensor data processing has become a significant engineering problem. At present, fault diagnosis models heavily depend on domain experts to manage the datasets of each task and create specific models for training. This approach is not only laborious but also challenging to sustain over time. Additionally, since the entire industrial process mainly relies on human intervention – including data processing and model selecting – it leads to long design cycles and high maintenance costs. Therefore, the efficient utilization of vast amounts of data for automated processing, coupled with the development of suitable models, has emerged as a critical concern that demands urgent attention.

The advent of the "BD-AI-IoT" era (Big Data, Artificial Intelligence, Internet of Things) has rendered manual formula adjustments insufficient to cope with the massive amounts of data generated. As a result, the era of fault diagnosis 1.0 emerged (FD 1.0), characterized by data-driven, automated processes using prediction-based, reconstruction-based, and hybrid approaches. In this automated process, fault diagnosis relies on data and can generate models from it. Various approaches have proven successful, including prediction-based (Chauhan & Vig, 2015; Z. Chen et al., 2022; Munir et al., 2019), reconstruction-based (Audibert et al., 2020; L. Li, Yan, et al., 2023; B. Zhou et al., 2019) and hybrid approaches (Bashar & Nayak, 2020; Han & Woo, 2022; Zhao et al., 2020). Despite their success, these methods face challenges in interpretability and depend heavily on data quality. To address these challenges, researchers have proposed incorporating data-driven domain-specific rules to improve interpretability throughout the process and reduce dependence on data quality (Xin et al., 2022), marking the beginning of the fault diagnosis 1.5 era (FD 1.5). Common strategies

* Corresponding author.
E-mail addresses: 1229080@mail.dhu.edu.cn (Q. Zhang), 2222778@mail.dhu.edu.cn (C. Xu), jie.li@dhu.edu.cn (J. Li), 1222014@mail.dhu.edu.cn (Y. Sun), bao@dhu.edu.cn (J. Bao), dan.zhang@polyu.edu.hk (D. Zhang).

https://doi.org/10.1016/j.eswa.2024.125861
Received 18 October 2023; Received in revised form 14 October 2024; Accepted 18 November 2024
Available online 22 November 2024

include the fusion of expert systems and neural networks, and the fusion of knowledge maps and deep learning (Chai, 2020). While these methods enhance interpretability, they also have limitations, such as incomplete fault knowledge rules and difficulty solving new problems.

The FD1.0 era was defined by data-driven methods that faced challenges in interpretability and model reusability. To address these issues, FD1.5 combined data-driven techniques with domain-specific rules, improving model interpretability but still relying on extensive human involvement. The emergence of LLMs has ushered in the FD2.0 era, where task-driven, autonomous fault diagnosis is possible. In task-driven fault diagnosis, the user provides the LLM with time series data and tasks, marking the beginning of the fault diagnosis 2.0 era (FD 2.0). The LLM systematically breaks down user tasks, integrating expert knowledge to handle each sub-task. This method enhances LLM to manage the entire task flow effectively (Ruan et al., 2023). The emergence of LLMs such as GPT3.5, GPT4, and Chat GLM (Du et al., 2022) which exhibit cognitive competence through perceptual learning, has produced impressive results across several domains. These generative models, supported by cognitive competence through perceptual learning, provide users with responses that meet their expectations. Fig. 1 outlines the progression from FD1.0 to FD1.5 and finally to FD2.0, highlighting the transformative nature of this process. This evolution demonstrates significant improvements in interpretability, automation, and task-handling capabilities. FD2.0, powered by LLMs, offers enhanced flexibility, reduced human intervention, and the ability to leverage both data-driven insights and domain-specific knowledge, marking a paradigm shift in fault diagnosis methodologies.

The advancements in LLMs offer substantial possibilities for transforming industrial fault diagnosis. LLM can define and decompose industrial tasks, simplifying steps such as time series data pre-processing, creation of fault detection/prediction models, interpretation of model outcomes, and identification of root causes (Tu et al., 2023). LLMs efficiently plays various roles in the fault diagnosis process and their performance in processing data after task decomposition is comparable to human performance (Cheng et al., 2023). LLMs, such as MetaGPT (Hong et al., 2023), HuggingGPT (Shen et al., 2023), and AutoGPT (Yang et al., 2023), are intelligent tools developed by humans, capable of autonomously completing assigned tasks and providing feedback or suggestions. Users can query LLMs based on their outputs, enabling continuous learning and improvement. This collaborative process allows LLMs to handle similar or new problems more effectively. As intelligent systems become more integrated into society, the demand for effective human–machine interaction increases. Therefore, the importance of machines producing reliable and readable information has become more crucial. Incorporating human-in-the-loop learning enhances flexibility, adaptability, and interactivity, improving machine capabilities.

Prompts are essential for LLMs to comprehend input information efficiently. The selection of prompts influences the model's problem-solving orientation and strategies, and must be customized to the specific task. Researchers from Peking University and Alibaba proposed a technique for classification and forecasting on time series data using LLMs. The method employs soft-prompting to match up time series based on distinct features such as frequency, shape, and value, which can enhance LLM's ability to handle time series embedding for diverse

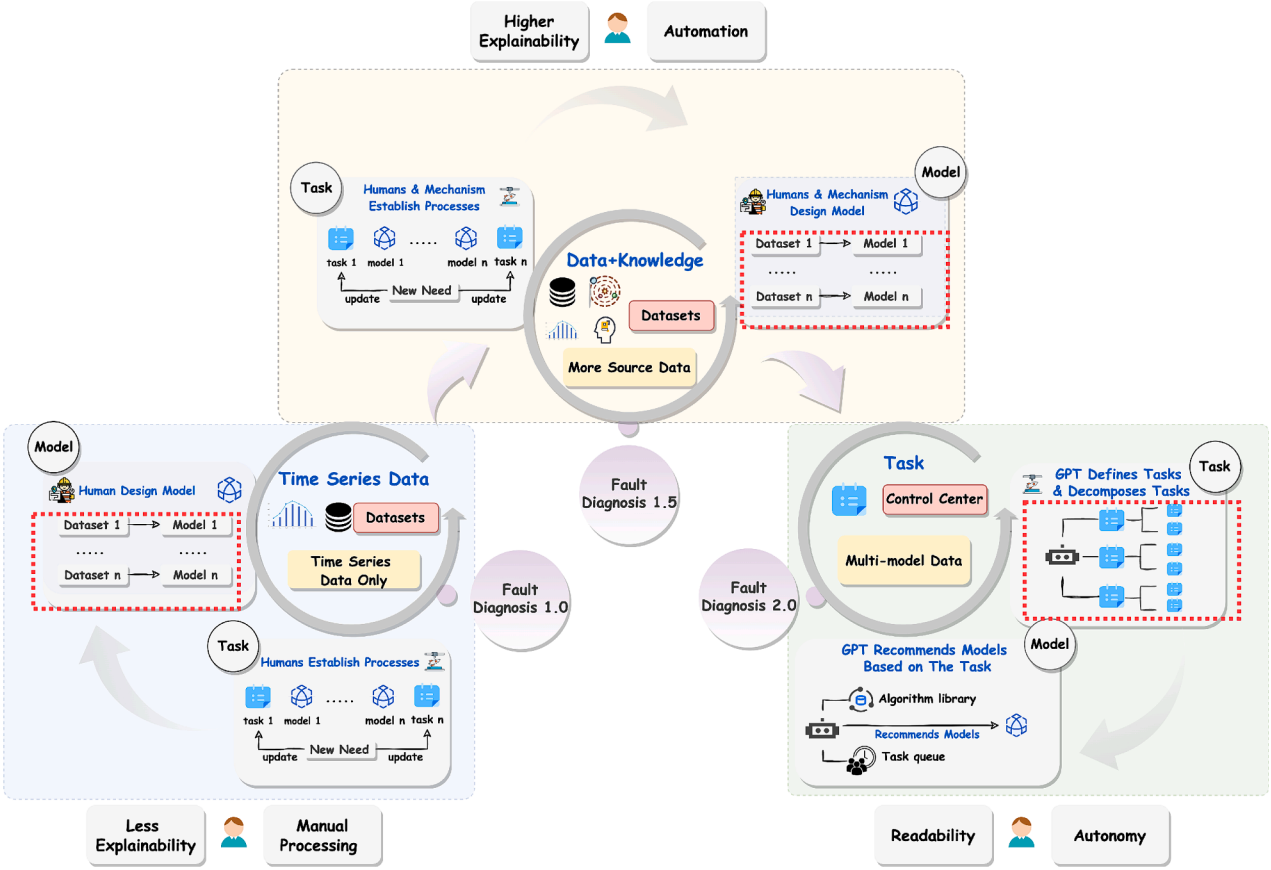


Fig. 1. The Fault Diagnosis Era has three stages: FD1.0, FD1.5, and the merging FD2.0. During FD1.0, the focus was only on one type of industrial time series data, requiring manual data labelling and code writing for each process. Additionally, the trained models lack interpretability. In the FD1.5 stage, the industry considered more types of time series data and increased knowledge to enhance interpretability. The emergence of learning methods such as transfer learning has led to increased automation throughout the entire process. With the emergency of LLM, fault diagnosis is transitioning to the FD2.0 stage. Users can drive LLMs by simply publishing tasks without the need for manual processing. LLMs autonomously empower AI by allocating different models for various tasks. Furthermore, the ability to interpret diagnostic results has enhanced their readability.

time series tasks. However, this pre-defined categorization limits flexibility in handling complex time series data, as it may not capture all key information or the spatial relationships between variables. LLMs face several challenges:

(1) Controlled generation: Achieving controlled generation in LLMs is challenging due to inherent instability, resulting in inconsistent answers to the same question. This research introduces prompt engineering to deal with time series data for fault diagnosis, thereby improving the model's ability to follow the user's instructions.
(2) Efficient adaptation: Adapting LLMs to vertical domains involves prompt learning and fine-tuning of model parameters (delta tuning). The goal is to equip LLMs with problem-solving capabilities within specific domains. In industrial fault diagnosis, it is crucial to determine how LLMs can replace conventional processes to promote efficient human–machine collaboration. Establishing an appropriate framework is essential to leverage LLM capabilities for efficiency and effectiveness.

This paper analyzes fault diagnosis tasks based on time series data in an industrial environment. Currently, the industrial anomaly diagnosis processes are highly complex and intricate, making fault detection and resolution challenging. Handling a wide range of parameters and variables, along with uncertainties in the data, adds to the complexity to these tasks. Traditional fault diagnosis methods often fail to manage unknown faults effectively and rely heavily on expert experience, resulting in a diagnosis process that is both time-consuming and difficult to automate. This paper aims to address industrial time series fault diagnosis as the task objective for a large language model (LLM). The proposed approach breaks down complex task objectives into a list of prioritized sub-tasks, thereby clarifying the objectives and enhancing the understanding of the LLM. This paper focuses on developing an LLM-based method for human-in-the-loop fault diagnosis of industrial time series data. Our research includes the following contributions:

(1) This research proposes an LLM-based method for developing a task-driven autonomous fault diagnosis process. This framework enhances human–machine interaction through a structured human-in-the-loop model.
(2) To overcome the limitations of predefined temporal features in processing time series data, this research proposes a combinatorial reasoning approach. This method identifies the roles of the LLM, incorporates few-shot learning, and integrates domain knowledge.
(3) Roles of the LLM in fault diagnosis tasks:
   i. Data processing: the LLM acts as a pipeline manager, managing the entire data pipeline and coordinating data flow and dependencies across various stages based on user input.
   ii. Causality modification: the LLM acts as a supervisor, accessing and correcting causal relationships inferred by algorithms using domain knowledge.
   iii. Fault prediction and detection tasks: the LLM functions as a controller, using models from the algorithm library and user requirements to make recommendations. It calls the local algorithm interface to execute results.
   iv. Fault diagnosis: the LLM acts as a decision-maker, using fault knowledge from the rule base to infer the cause of detected anomalies. If no cause is identified, the LLM conducts a network query to gather information and provide a solution.

2. Related work

2.1. Large model vs small model

With the rapid development of LLMs, several generative models with autonomous cognitive capabilities have emerged, including GPT3.5/4, ChatGLM, and Vicuna. These models with billions of parameters are classified as large models, while those with millions of parameters are classified as small models (S. Li et al., 2022). Large models have shown remarkable performance in reasoning ability, computational power, and generalization compared to small models (L. Li, Zhang, et al., 2023; Shridhar et al., 2023). Although the emergence of large models has changed the decentralized model development mode where multiple tasks within a single AI application scenario require the support of multiple models, each requiring algorithm development, data processing, model training, and tuning (Y.-F. Li, Wang, et al., 2023). Nevertheless, small models still offer notable advantages such as low training costs, smaller size, and easy maintenance. Therefore, a combined approach using both the 'small model workshop' and 'large model industrialization' can be adopted to exploit the full potential of these models taking into account their respective advantages.

The generative capability of LLMs is the most notable feature. In industry where missing data is inevitable, LLMs prove their value in generating missing event series values or filling in time series data, enabling domain models to effectively address the challenge of missing data. In addition, LLMs serve as a comprehensive model library that integrates different domain models. This integrated approach is achieved by aggregating the multi-models using a scoring mechanism to identify the optimal model.

Domain models are more suitable for specific domains, where LLMs are more generalization to handle more complex industrial tasks. Yao (2023) argues that large models and small models can combined from three aspects: data interaction, model interaction, and application interaction, as shown in Table 1.

This combination of small and large models enhances the ability to learn and handle complex problems while laying the foundation for deep feature extraction and processing of time series data. These models can discover underlying discover patterns, trends, and correlations, extracting this information as input features for domain models. Consequently, domain models can learn directly from the fine-grained features, reducing their processing of the raw data and improving the ability of domain models to understand and analyze time series data.

2.2. LLM & time series data fault diagnosis

The success of ChatGPT has attracted considerable attention from academics, leading to increased research interest in the application of LLMs to fault diagnosis of time series data (Wu et al., 2023). Li et al. (2023) have identified device health management and prediction as a future research direction for the use of LLM. They argue that current deep learning models need to move from unimodal, single-task approaches to multimodal, multi-task methods that exploit large amounts of data. They believe existing deep learning methods have limitations in terms of cognitive and generalization capabilities. Similarly, Zhou et al. (2023) propose an innovative solution called D-Bot, designed

Table 1
Small model workshop vs large model industrialisation (Ma et al., 2023; Q. Yao, 2023).

Type of Tasks	Large Model Industrialisation	Small Model Workshop
Data Interaction	Generate large, high-quality corpora for small models training	A corpus of small models to complement large models training
Model Interaction	The large models guide the small models' training	The small models distil the large models in reverse, using the small models to make sample value judgments to speed up the convergence of the large models
Application Interaction	The large models act as a dispatch center and call the small models	The small models learn from large models to improve their skills

specifically for database administrators. D-Bot uses LLMs to continuously acquire database maintenance experience from textual sources and provide continuous diagnostic and optimization recommendations for target databases. In addition, Chen et al. (2023) proposed a method called RCACopilot, which uses LLM analysis to identify the root causes of cloud accidents automatically. RCACopilot relies on LLM to summarize failure information and predict multi-event failures to identify their root causes. Ahmed et al. (2023) apply LLMs inference and summarization techniques to historical failure texts such as failure descriptions, root cause reports, and recommended solutions. Through this approach, they recommend root causes and solutions for emerging problems. This research proposes a task-driven framework using LLMs for fault diagnosis of time series data. The framework combines LLMs with Pandas, Scikit-learn, SQL, and local databases for data processing and model recommendation calls on time series data. In addition, these studies demonstrate the promising applications of LLMs in time series fault diagnosis, overcoming the limitations associated with existing approaches.

2.3. Development of time series data prompt

The prompt is a critical interface for users to interact with LLM. Without it, LLMs cannot understand the user's problem. A prompt consists of two components: an instruction that tells LLM what to do, and a question that specifies what the user wants LLM to do. Teven et al. (2021) have pointed out that prompts contain a considerable amount of information, and that one prompt can be equivalent to 100 real data samples. In addition, prompts provide significant support in data-sparse vertical domains and perform admirably even in zero-shot scenarios, making prompt technology an essential component in the use of LLM.

Currently, prompt design methods are classified as few-shot (Madotto et al., 2021), chain of thought prompting process (CoT) (Wei et al., 2022), zero-shot chain of thought (Kojima et al., 2022), self-consistency (Wang et al., 2023), knowledge prompt generation (X. Chen et al., 2022), procedural assisted language (PAL) (Gao et al., 2023), and ReAct (S. Yao et al., 2023). However, prompt design methods for time series data are still at an exploratory stage. Li et al. (2022) proposed a prompt technique for predicting time series features, but the process of using language to drive Bert's model for predicting time series needs to consider the problem of aligning the time series data. In addition, the prediction of future information required the consideration of domain knowledge for better results.

To address these issues, Xue et al. (2023) developed the PromptCast method, which uses text-to-text input for time series prediction. However, this approach missed a significant amount of information in the time series data. Consequently, Sun et al. (2024) proposed the TEST approach, which converts time series data into a textual representation described in terms of frequency, shape, and value. This enables LLMs to process and analyze time series data. This research has designed a prompt structure for time series fault diagnosis capable of processing time series data, model retrieval, causality correction, and other applications, and the results demonstrate its effectiveness.

3. LLM-based fault diagnosis 2.0 framework for steel metallurgy driven by human-in-the-loop tasks

3.1. LLM-based task-driven method

The traditional fault diagnosis process for industrial time series data relies heavily on manual intervention. This includes manual labeling and pre-processing of the data, manual integration of domain knowledge to correct causal relationships, manual selection of algorithm models for data prediction and anomaly detection, and manual integration of domain knowledge to determine the cause and resolution of failures. A significant amount of human and physical resources is essential throughout the fault diagnosis process. However, this extensive manual intervention consumes a lot of human and material resources, and non-professionals handle fault problems effectively. In terms of timeliness, there is a notable delay, making a quick diagnosis. Even professional operation and maintenance staff have limited knowledge and may struggle to solve new problems that arise. Therefore, a method is needed that can automate task processing, store extensive domain knowledge, and handle complex tasks effectively. LLMs facilitated rapid problem-solving through user questions and answers, acting as a link between users and the knowledge base. By continuously adapting its role in response to user requirements, the LLM captures the most relevant knowledge and contributes to an automated workflow. This paradigm shift in data and knowledge flow has revolutionized current task-based processes. We have redefined the LLM-based task-driven approach and compared it with two other driving methods: data-driven and data and knowledge-driven. The data-driven approach relies solely on data to drive the entire process, while the data and knowledge-driven approach incorporates domain knowledge to augment the data-driven approach. In contrast, the LLM-based task-driven approach encapsulates both data and knowledge to define and decompose user-proposed tasks. The resulting flow of data and knowledge follows the flow of tasks. This approach employs a hierarchical task decomposition strategy. Firstly, it deconstructs complex tasks into more straightforward, manageable ones. These simpler tasks are then broken down into a series of sequential steps. Each step has a clear objective and expected output, linked in a specific order and logical relationship to form a complete process. This approach achieves task-driven autonomy, enhancing the precision and efficiency of the entire task execution.

3.2. Proposed framework——industry time series fault diagnosis based on large language model (LLM-TSFD)

The framework for human-in-the-loop task-driven time series fault diagnosis using LLM is shown in Fig. 2. The model relies on two primary inputs throughout the process: the user prompts and the industrial time series data from the intermediate layer. The user prompts consist of a series of instructions and questions. Once processed by the human-in-the-loop LLM, the desired results can be output. In the data processing task, the input data is visualized and the pre-processed data is stored locally in CSV format. In the fault detection task, the output is the corrected causality. In the fault diagnosis task, the model recommends prediction/detection models and provides the results of fault detection and prediction. During this process, the LLM searches the knowledge base for the most relevant content as output. When encountering a new problem, the LLM can search the network to find an appropriate solution.

In Fig. 2, the human-in-the-loop LLM is shown on the right. The processing flow of the human-in-the-loop LLM follows a "loop" mechanism, where the user defines a goal, uses the LLM to understand and decompose the task, and then executes the task in the order of the decomposed tasks. After completing the task, the LLM interacts with the user in two ways. First, through binary user feedback, the LLM provides an answer, and the user can respond as either 'satisfied' or 'dissatisfied.' If dissatisfied, the LLM revises the answer or the user modifies the question to guide the LLM towards a more accurate response. This iterative process helps produce a more refined answer. Second, by quantifying user feedback. During task execution, the LLM queries the content of the database or vector database. The similarity between the queried content and the knowledge base determines the confidence level. A low confidence indicates a weak match, while a high confidence indicates a strong match. If the confidence is low, the LLM notifies the user that the answer may be imprecise and prompts the user to provide a more detailed description by modifying the question. Throughout this task-driven process, there is constant interaction between the human and the LLM, encouraging feedback at different stages of the cycle. This feedback during the loop improves the performance and interpretability of the LLM.

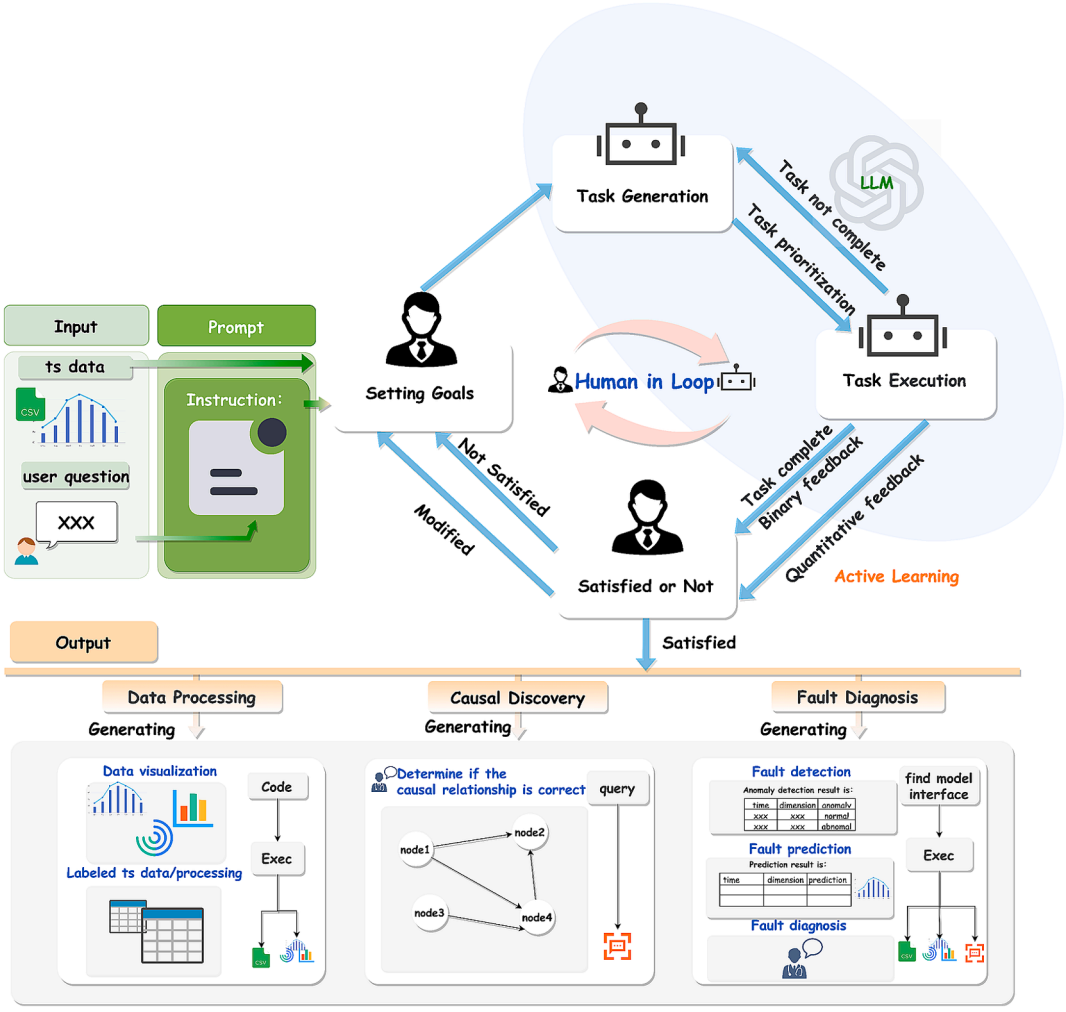


Fig. 2. A structural framework for fault diagnosis using LLM outlines the entire process, from input to output, utilizing LLM. Inputs consist of user requirements and data to be processed, combining prompts to form a complete problem that becomes the user requirement goal. The user requirement is transferred to LLM for decomposition into individual subtasks and sequencing. Tasks are executed sequentially based on their ranked sequence. The results are provided to the user in two forms: quantitative feedback and binary feedback. If unsatisfied with the results, users can modify them as standard answers and pass them back to LLM for learning purposes before receiving a satisfied output. Finally, the results are integrated with third-party tools for applications in data processing, causal discovery, and fault diagnosis.

The working mechanism of LLM is reflected in the task decomposition and execution. This process is mainly divided into three modules: data processing, causal discovery, and fault diagnosis, as shown in Fig. 3. The section framed by the red dotted line highlights the working part of LLM. For data processing, the user input question undergoes token embedding and position embedding to produce a vector X, $X = [x_1, x_2, x_3, \ldots, x_n]$, n denotes the nth word. The LLM processes the X vector to generate an answer. The input vector X is multiplied by the W_Q, W_K, and W_V matrices, which are obtained by the model, and produce the Q, K, and V matrices. These matrices are employed by the self-attention mechanism to compute similarity, as shown in Equation (1). Q_i, K_i, and V_i in Equation (1) correspond to the query vector, key vector, and value vector for the ith element, respectively.

$$Q_i = W_Q \cdot x_i, K_i = W_K \cdot x_i, V_i = W_V \cdot x_i \tag{1}$$

Perform a dot product operation on the query (Q), key (K), and value (V) matrices. Next, multiply and sum the attention scores to obtain attention weights for contextual semantic vectors. The complete procedure can be expressed using the following Equation:

$$score(Q_i, K_j^T) = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \tag{2}$$

$$w_{ij} = softmax(score(Q_i, K_j^T)) \tag{3}$$

$$Z_i = \sum_{j=1}^{n} w_{ij} V_j \tag{4}$$

For each element x_i and x_j in Equation(2), we calculate an attention $score(Q_i, K_j^T)$ which represents the degree of attention of x_i to x_j, where d_k is the dimensionality of the key vector, and $\sqrt{d_k}$ is the scaling factor; the attention *score* in Equation (3) is obtained by the *softmax* function to get the w_{ij} attention weight; V_i in Equation(4) is the value vector, which is weighted and summed to get the final context matrix Z_i. The context matrix contains not only the information of the current word but also the contextual information, helping the model to understand the meaning of each word in different passages.

The LLM maps the context vector to a higher dimensional space using a fully connected layer. It then maps the result to a vocabulary-sized vector, which is converted to provide a probability distribution for each word using the *softmax* function. The output code meets the user's requirements. For causal discovery, the causal relationships entered by the user are embedded and then matched for similarity with the contents of the vector database storing the expert knowledge to find the segments

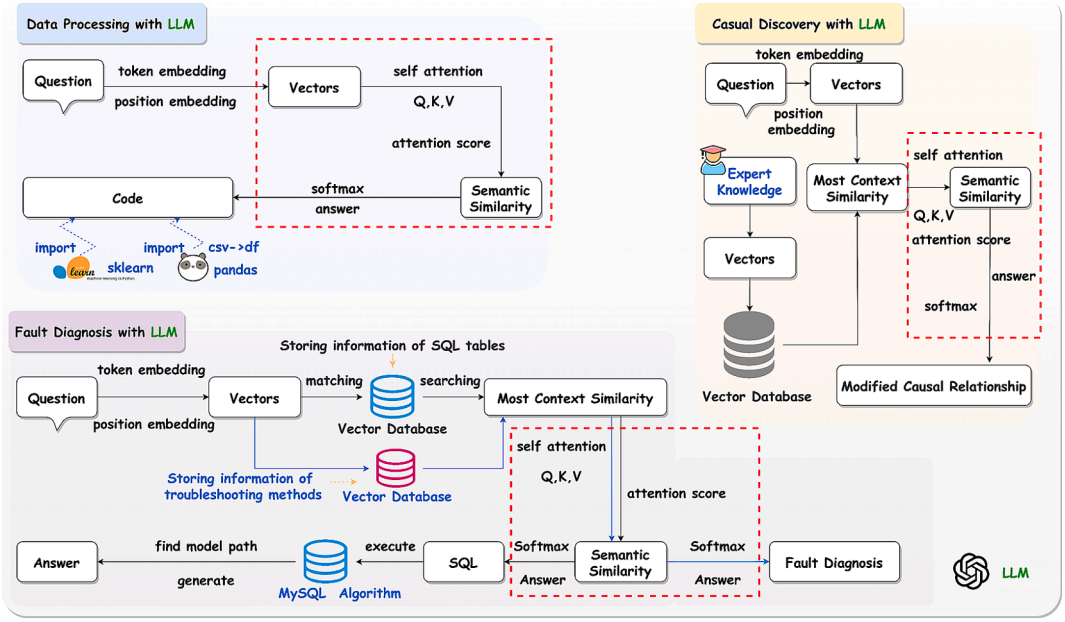


Fig. 3. Schematic diagram of task processing using LLM: The LLM is a transformer structure depending on Q, K, and V as key parameters in the attention module. The primary objective of task processing in LLM is to vectorize input content and match it in the knowledge base for the most similar output generation.

with the highest similarity. These segments are fed into the LLM to generate context vectors using the self-attention mechanism, similar to the process of generating answers in data processing, and ultimately produce the modified causal relationships.

For fault diagnosis, the user's query is embedded into vectors and compared with the content in the vector database containing the SQL database's table structure. By identifying the most similar vector segments, the LLM uses self-attention mechanisms to establish contextual semantic similarity. Ultimately, an SQL statement is generated according to the user's requirements. This SQL query operates on the MySQL database to determine the position of the model and propose the appropriate model to the user. During fault analysis, the blue arrows indicate the process after text embedding. After encoding, the vector database is searched for the cause of the fault, and the most similar segment is found. The self-attention mechanism is then employed to compute the context vector, followed by generating a diagnosis of the fault's cause.

4. The roles of LLM in industrial time series data fault diagnosis

The LLM serves as a universal model, making it crucial to find its ability to industrial production and drive the industry forward. In interactions with LLM, various individuals and contexts can interpret the content differently. Therefore, establishing rules for the LLM that define roles, scenarios, and other relevant factors is necessary to enhance the model's domain knowledge and improve outcomes. AutoGPT is an extension of ChatGPT that allows users to define roles and assign them to GPT. Conversely, MetaGPT allocates specific roles and provides detailed information, improving the LLM's understanding of their respective domains. Therefore, assigning appropriate roles to LLMs is essential for effective learning in industrial time series data. Fig. 4 depicts the roles expected from LLMs during the FD2.0. These roles are as follows:

(1) Data pipeline manager: the LLM carries out automated data preprocessing, cleaning, labeling, and visualization. This reduces the time and effort spent by operation and maintenance staff on data labeling and processing.
(2) Supervisor: the LLM corrects causal relationships established by traditional algorithms. LLM achieves this task by integrating domain knowledge and merging it with logical reasoning to achieve precise determinations of causal relationships.
(3) Controller: the LLM calls on the models from the algorithm in complains with user suggestions, to forecast and identify faults. In response to actual production issues, LLM can suggest appropriate machine learning models and initiate their execution locally.
(4) Decision-maker: the LLM detects abnormal outcomes by comprehending the detection process and presenting the model's behaviour in a manner that can be understood by operational and maintenance staff. Moreover, LLM outlines the diagnosis results for staff to reference and understand.

Each of the roles played by LLM is described in detail below.

4.1. Pipeline manager

Current industry domain models rely on data quality. The data originates from several sensors, devices, and systems. Nonetheless, this data often contains noise, missing values, and outliers, which can affect the accuracy and credibility of the models. Additionally, industrial data has unique structures, formats, and standards. To analyze this diverse range of data efficiently, it is crucial to integrate and unify the analysis of these heterogeneous data sources. Traditionally, professional staff were necessary for processing, labeling data, and incorporating the results with relevant professional knowledge for efficient interpretation. Moreover, specific platform interfaces may restrict the flexibility of user preferences. This research proposes the use of LLM to process time series data. Firstly, employing direct data analysis through dialogue reduces the need for manual design of data analysis techniques, minimizes subjectivity, and enhances work efficiency. Secondly, using dialogue to output data charts in the user's desired format effectively meets specific requirements.

The LLM serves as a pipeline manager, overseeing the data flow and processing. The overall flow is shown in Fig. 5. Users provide an industrial time series data file and a task question as inputs to the pipeline manager. Subsequently, the pipeline manager calls the LLM code generation agent, which generates the required code based on the user's inputs. The generated code is then executed and returns an output result.

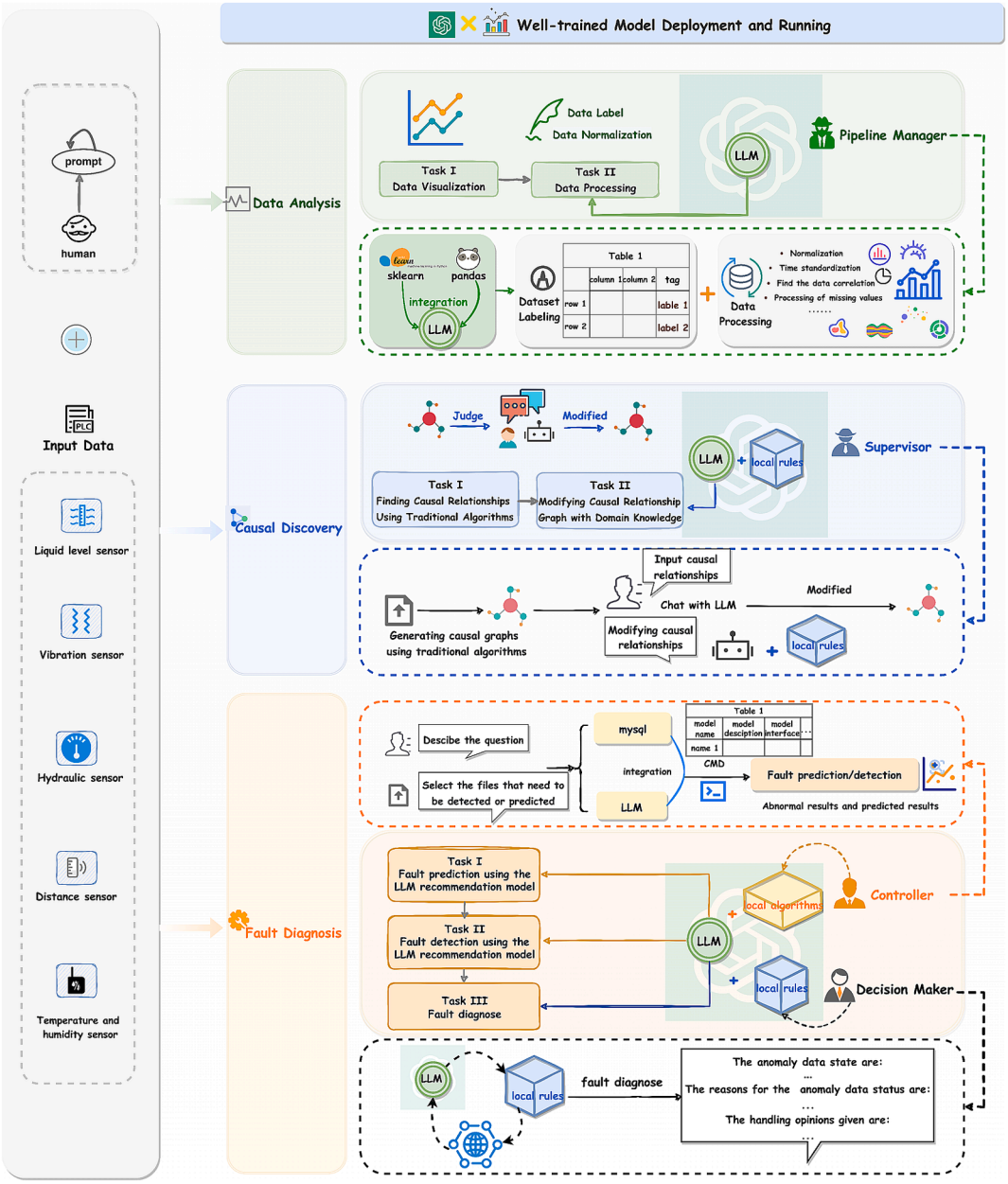


Fig. 4. Value of LLM's role in fault diagnosis 2.0: For data processing, LLM acts like a Pipeline Manager. For causal discovery, LLM functions as a Supervisor. Lastly, for fault diagnosis, LLM assumes both the Controller and Decision Maker roles to effectively diagnose faults.

In cases where data preprocessing is required, LLM generates the processed CSV file and saves it locally. Similarly, if data visualization is needed, visual charts are generated to meet user's needs. During the code generation stage, the pandas and sklearn libraries are imported, allowing for the use of their respective functions for data processing. Furthermore, if LLM detects changes in the dimensions of the input data, it will save the historical information. As a result, models that rely on these dimensions are labelled as "requiring maintenance and updates.".

The LLM automatically generates the appropriate code for data analysis, considering the user's task requirements. To simplify processing operations, relevant third-party libraries are introduced and thus autonomously generate the results expected by the user.

4.2. Supervisor

Following data processing, it is necessary to select appropriate causal inference methods or models for causal discovery based on the specific engineering problems and data features. Once causality has been extracted through the model, the results need to be evaluated and validated to ensure their reliability and applicability. Traditional methods typically require a series of operations such as cross-validation and sensitivity analysis for verification. Additionally, discussions and validations in collaboration with domain experience are indispensable. LLM has certain reasoning abilities. When provided with model inference outcomes, expert knowledge, and requirements, the LLM can act as a supervisor, combining its reasoning with model outputs to judge and verify causal relationships. By integrating its reasoning outcomes with the model output, the LLM can correct and refine the causal relationships, resulting in more accurate conclusions.

The discovery of causality exhibits some deviation due to the lack of domain knowledge in the traditional model. To learn true relationships, relevant domain knowledge needs to guide the model. This research suggests that the LLM can serve as a supervisor, as shown in Fig. 6. Utilizing LLM's reasoning ability to modify the original causal findings

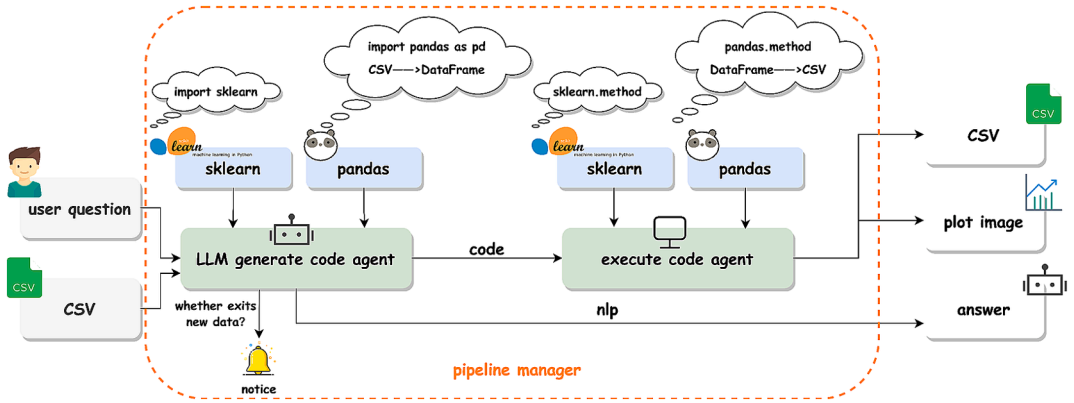


Fig. 5. LLM as pipeline manager for data processing: LLM combined with some Python libraries for data processing generates complete code content. Executing the code in the local environment then generates the desired answers, CSV files, images, etc.

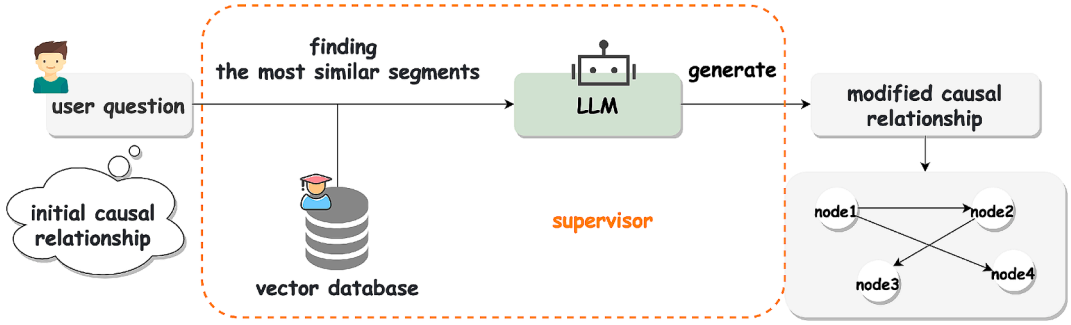


Fig. 6. LLM as supervisor for causal correction: The user inputs the initial causality, corrects the causality in conjunction with the expert knowledge base, and finally outputs the corrected result.

enhances the discovery of causal relationships. Users input the causality deduced from the traditional algorithm model and modify it by combining domain knowledge after entering LLM to deduce a more reasonable causality. Finally, the output of causality is expressed as a causality graph.

During the process, the user's initial relationships are embedded into vectors and compared for similarity with vectors stored in the vector database. The more similar segments are fed into the self-attention mechanism generating responses that meet the user's expectations.

4.3. Controller

In the enterprise, models are typically deployed in local environments to ensure privacy and security. When predictive maintenance, anomaly detection, or equipment remaining life prediction is needed, the appropriate model must be chosen. If the state of data remains unchanged (i.e. no new sensors or additional dimensions appearing), pre-trained models can be used for predictions. However, in real-world scenarios, incoming data is continuously updated, making it essential to maintain and update the model constantly to ensure its effectiveness and adaptability to evolving data.

This research proposes using LLM as a model controller to help users select models and provide feedback on whether the model needs to be updated or not, as shown in Fig. 7. The user inputs the problem and time series data to LLM, which then splits into two processing pathways. Firstly, it compares the input time series data file to check for new sensors or dimensions. If LLM detects any new dimensions, flag the relevant model and mention to the developer that models need to be updated. Secondly, if there are no new dimensions, LLM recommends a model from the algorithm library based on the user's requirements and processes the input file to generate the desired output.

In the process, LLM serves as a controller, managing the selection of models from the algorithm library while assessing the dimensionality of the input data. This approach quickly selects models for processing and

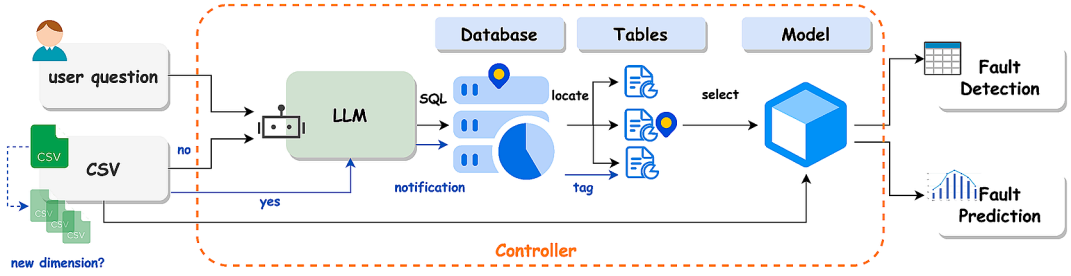


Fig. 7. LLM as a controller for model scheduling process: The user provides the system with the requirements and data to be processed. The LLM algorithm then matches this input against the SQL database to recommend the model with the highest matches. Subsequently, fault detection/prediction results are output by the recommended model.

supports model updating to ensure accurate prediction results.

4.4. Decision maker

The interpretability of machinery and equipment fault diagnosis in industry is often poor. Data-driven models make it difficult for users to understand the specific causes and solutions of faults. Similarly, knowledge-driven models exhibit incomplete knowledge representation (e.g., knowledge graphs and expert systems) making it difficult to deal with novel problems. This paper proposes using LLM as a communication bridge between user requirements and processing solutions. LLM can find an optimal solution by understanding user needs, employing networked search, calling models, and connecting to databases to facilitate user understanding.

This research proposes that LLM acts as a decision-maker with local knowledge bases and networked search to offer readable explanations of the cause of diagnosed faults, as shown in Fig. 8. Firstly, the user's question is embedded and matched with the domain fault knowledge stored in the vector database. If there are segments with relatively high similarity, a diagnosis result is generated. In cases where the similarity is low, a network query is conducted to obtain relevant information for the question. The answer is then stored in the vector database, supplementing the existing knowledge and enabling to generate the desired results for the user.

4.5. The design process of a fault diagnosis prompt based on industrial time series data

The reasoning process in a LLM comprises three main components: input, reasoning, and output. Prompting also serves as a guiding mechanism that directs the entire reasoning process in the LLM. Without a prompt is not designed, the probability $P(x)$ of the model predicting the text when the user inputs question x can be expressed as follows:

$$P(x) = \prod_{i=1,2,\ldots}^{n} P(x_i|x_1, x_2, \ldots, x_n), i \in n \tag{5}$$

The addition of a prompt can be seen as providing additional constraints. If the prompt is set to y, the probability of the model predicting the text is $P(x|y)$. This indicates that by restricting the user input question under the condition of instruction, the model can generate the desired result for the user. In essence, the prompt serves to narrow down the original probability space into a subspace associated with it. Thus, the predicted probability, $P(x|y)$, can be expressed as follows:

$$P(x|y) = \prod_{i=1,2,\ldots}^{n} P(x_i|y, x_1, x_2, \ldots, x_n), i \in n \tag{6}$$

In LLMs, the addition of a prompt modifies the probability space by providing conditional constraints that guide the model towards generating outputs that are conditionally relevant. This process enhances the alignment between the responses of the language model and users' expectations, thereby improving its performance and generalization capabilities. Furthermore, an accurate prompt plays a crucial role in making the LLM function effectively and produce satisfactory results. However, it is important to note that, the LLM, a generalized model, lacks domain-specific understanding. They require guidance within a domain of knowledge to know how problems are addressed. To address fault diagnosis in time series data within the industry, this paper proposes a prompt structure to guide task-driven development. The prompt design structure is shown in Equation (7).

$$\begin{aligned} P(x|y) &= \prod_{i=1,2,\ldots}^{n} P(x_i|y, x_1, x_2, \ldots, x_n), i \in n \\ s.t.y &= (T, R, I, TD, O, Rea, DK) \\ R &= \{R_1, R_2, R_3, R_4\} \end{aligned} \tag{7}$$

In Equation (7), constraints are added based on Equation (1). The constraint content, denoted as y, which is the structure of the prompt proposed in this research. The prompt consists of seven core components: time (T), role (R), input form (I), task decomposition (TD), output form (O), reasoning form (Rea), and domain knowledge (DK). The role (R) is a quaternion structure consisting of four roles proposed in this research: R_1,R_2,R_3,R_4. Specifically, R_1 represents an expert in time-series data analysis, R_2 serves as the modifier for time-series causality discovery, R_3 acts as an expert in time-series fault diagnosis model recommendations, and R_4 assumes the role of a time-series fault diagnosis decision expert.

In Equation (7), T indicates the current date for ease of recording. R and DK provide the LLM with specific roles and relevant industrial domain knowledge. This enables the LLM to understand what kind of scene it is in, what kind of role it is playing, and what kind of task it needs to complete next. A well-defined set of industrial scene and domain roles provides LLM with a clearer understanding of their work, similar to human work engineering. This allows the model to comprehend its position within a company and anticipate the next steps.

Next, the LLM needs to be configured to accept input in the form of I and output in the form of O. To generate code based on a user's task, it may be necessary to process uploaded files by users into a format that can be understood by code. To enhance the data processing capabilities of the LLM, tools such as Pandas can be integrated, converting input into a DataFrame format for programmatic processing, which increases efficiency in data handling.

Given the user's task requirements are sometimes unclear, it becomes necessary to automate task decomposition. This process enables LLM to break down and comprehend the user's purposes incrementally, thereby achieving the expected outcomes. Additionally, during reasoning the method Rea sets as follows: LLM gradually think step by step which ultimately leads to better results generation.

The LLM operates based on the above framework and then outputs the results. The process of data analysis and processing by LLM is shown

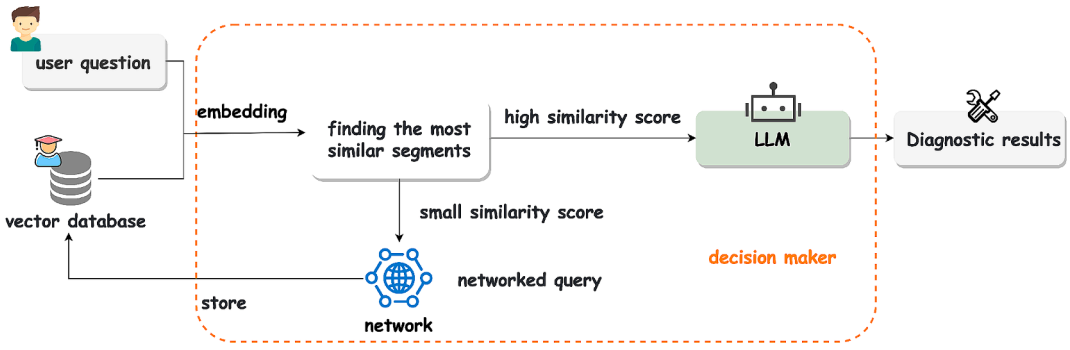


Fig. 8. LLM as decision maker for decision making: The user enters a requirement that is encoded and matched with the content in the vector database. The most similar content is then output. If there is no match, the system searches the network and adds any found content to the vector database as new knowledge.

in Fig. 9. The first step involves introducing the necessary toolkit, followed by determining the inputs for the analysis process. Finally, the user's task requirements are decomposed and executed. Upon local execution of the code generated by LLM, an explanation of the running results is returned simultaneously to facilitate user comprehension. Table 2 demonstrates an example architecture for time series fault diagnosis prompt design.

5. Case study

Iron and steel metallurgy is a crucial industrial domain that heavily relies on efficient equipment to maintain productivity and product quality. Currently, an iron and steel smelting enterprise located in Baoshan District, China, is facing challenges in dealing with time-series data fault diagnosis, especially in terms of effective resource use and work efficiency. Therefore, there is an urgent need for an intelligent and autonomous approach to enhance development cycles, post-maintenance processes, and user readability.

5.1. Establishment of the framework under environment

To address the problems arising from FD1.0 and FD1.5, this paper proposes the task-driven approach of LLM for fault diagnosis applications in the field of iron and steel metallurgy. Fig. 10 illustrates a continuous casting equipment fault diagnosis scene.

The platform is divided based on the roles established by the LLM. The final fault diagnosis module includes two key components: selection of the fault model and question-and-answer of diagnostic information, encompassing both roles assigned to the LLM. The first module of the LLM system serves as a data pipeline manager. The user uploads a CSV file containing data from a sensor on a mechanical device during a specific period and specifies the task to be processed to obtain the corresponding output. In contrast to traditional approaches that require manual experience for designing data processing methods and domain staff with knowledge of relevant upper and lower limit values for tagging data, the LLM can replace these tasks. The user inputs the task into LLM, which then decomposes it into subtasks, provides feedback on tuning code, executes locally, adds relevant domain experience for labeling based on this experience, and allows users to determine their desired output format. Users can visualize input data in any format by informing the LLM of the type of display they prefer. Additionally, using historical information as reference points, the LLM determines whether new dimensional data is present in current input files; if so, subsequent models are tagged accordingly, with notifications regarding maintenance or updates. Fig. 11 shows the user dialogue process when performing operations related to processing equipment-related CSV files within iron and steel metallurgy fields using LLM's visual display capabilities. If users are unsatisfied with results generated by LLM's initial outputs they may provide feedback which will prompt further responses from LLM until satisfactory results are achieved.

The second module of the system is causal discovery, where the LLM serves as a corrector of causal relationships between variables in the sensor data. Initially, users establish basic relationships between variables using common causal analysis algorithms. The outputs from these algorithms are then fed into the LLM, which combines domain knowledge learned from a rule base to determine whether any relationship needs to be corrected. While traditional methods require users to possess or search for relevant knowledge to make causal judgments, after employing an LLM, allowing for replacing domain experts with automated processes. Fig. 12 illustrates how causality correction operations are performed using an LLM through binary and quantitative user feedback mechanisms. In binary feedback mode, users can indicate their satisfaction with the LLM's answer by clicking "agree" or "disagree." In quantitative feedback mode, the LLM assesses confidence levels based on similarity scores with matches in the vector database; if confidence scores are too low, proactive feedback is provided to prompt more detailed problem descriptions from users.

The third module is fault diagnosis, involving both the LLM controller and the decision-maker. In traditional approaches, a professional operator designs a model and provides a description for users to select, including inputs and parameters that require modification for each use. However, with LLM, users only need to describe the task, and the LLM matches their description with the most suitable model from the local algorithm library. The user then selects input data for running through the model interface before obtaining the desired results as output. Fig. 13 illustrates this process. The fault diagnosis module comprises three sub-modules: recommending fault detection models, recommending fault prediction models and making fault diagnosis suggestions. When suggesting models for detecting or predicting faults, the LLM retrieves a database to recommend an appropriate model based on user requirements. If users are unsatisfied with the recommended results, they can provide binary feedback (satisfied/unsatisfied). During fault diagnosis, the LLM searches the database for similar segments that may provide answers. If none are found, it indicates low similarity between user questions and knowledge base content, requesting more detailed problem descriptions. Additionally, four dialogue options are available per answer: Copy; Quote; Query; and Evaluate. 'Copy' allows users to copy an answer to a clipboard. By selecting "Quote", users can diagnose across lengths. By choosing a specific response from the context, quoting it, and then posing a question, both the quote and the

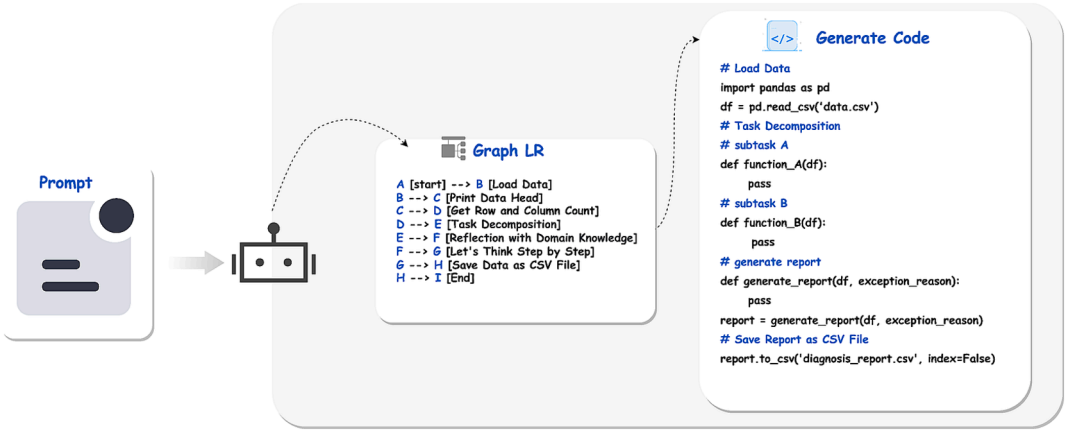


Fig. 9. The process of generating code for prompt LLM with an example of data processing: The user's requirements are analysed and broken down into sub-requirements under the guidance of a prompt. LLM prioritizes and processes these sub-requirements accordingly. If the issue pertains to data processing, the final output of the LLM would be the corresponding code for handling such data.

Table 2
Time series data fault diagnosis prompt design framework.

T: Today is {today_date}.
R*₁ & *I* & *O:If you are a time series data analysis expert and you have provided data in the format of a pandas dataframe. The data can be displayed and output in CSV format. dataframe (df), 'print(df.head({rows_to_display})) ': {df_head} The dataframe is in pandas dataframe format with {num_rows} rows and {num_columns} columns. The first row represents the header. ***TD***:If you are in an industrial setting, try breaking down this task into several subtasks. ***Rea***:Please think step by step. ***DK***:Common questions: **Input User Question**: The task provided by the user is:

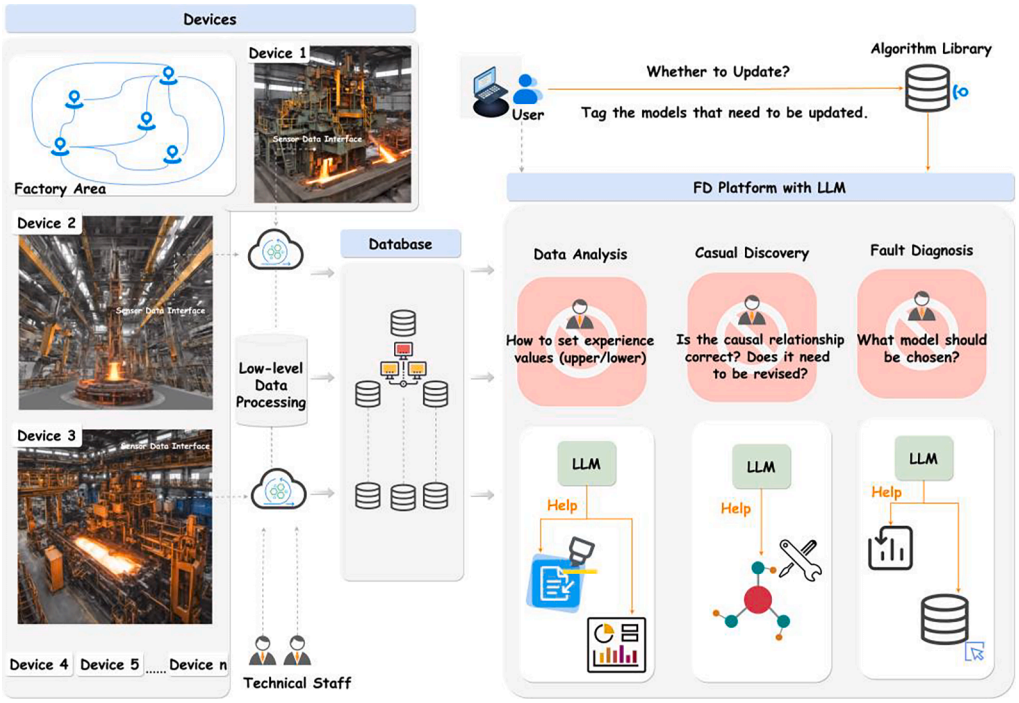


Fig. 10. LLM-based fault diagnosis platform for industrial time series data: Implementing the LLM has resulted in not only a reduction of both human and material resources but also a decrease in the development cycle of the platform. Additionally, it effectively utilizes empirical knowledge to assess the accuracy of causality and the appropriateness of selected models for facilitating decision-making processes.

query can be transmitted into LLM for further analysis and response. Fig. 13 displays these options along with network search capabilities ('Query') when quoting previous replies from LLM or providing binary feedback ('Evaluate').

5.2. Discussion of the outcomes of the LLM-TSFD

5.2.1. Comparison of LLM-TSFD and steel metallurgy fault diagnosis methods without the use of LLM

This paper conducts a comparison between the proposed method and a method that does not use the LLM. Specifically, a platform for fault diagnosis in steel and metallurgy companies is developed. This platform, based on deep learning methods, does not incorporate the LLM, as demonstrated in Fig. 14.

The platform utilizes pre-trained deep learning models for fault analysis. The user selects the appropriate model, and the model detects anomalies. Additionally, users can choose a prediction model that predicts values in future time windows and detects anomalies based on predefined rules (as shown in Fig. 14). Generally, deep learning models used for fault diagnosis and prediction in industry are typically supervised or semi-supervised learning models. These models necessitate manual labeling of data to achieve precise detection and prediction. However, in situations where labeled data is not readily accessible within an enterprise, unsupervised learning may be the sole feasible alternative for model training. Additionally, new data patterns often emerge in industrial time-series data that cannot be learned from historical data alone. Therefore, there is a need to improve the capability of unsupervised models. One potential solution to the challenge of data labeling is the utilization of LLMs, which can offer advantages in terms of both time and efficiency. By relying on LLMs instead of manual labeling processes, organizations may be able to streamline their workflows and achieve more accurate results with less effort.

Deep learning models need labeled data for training. Given the complexities of industrial time series data, it is essential to enhance multidimensional anomaly detection based on highly correlated data. Operators can use causal algorithms to identify causal relationships between each dimension. However, relying solely on these algorithms might result in erroneous relationships. For a more accurate causal relationship, merging LLMs with expert knowledge bases can help make more correct judgments.

Fig. 14(a) illustrates the platform's model configuration module. The backend provides interfaces for ten pre-trained models for anomaly detection and prediction. To detect anomalies effectively, users must select an appropriate deep learning model. However, since they lack knowledge about which model would yield better results, they can only achieve optimal outcomes through continuous trial and error. Fig. 15(a)

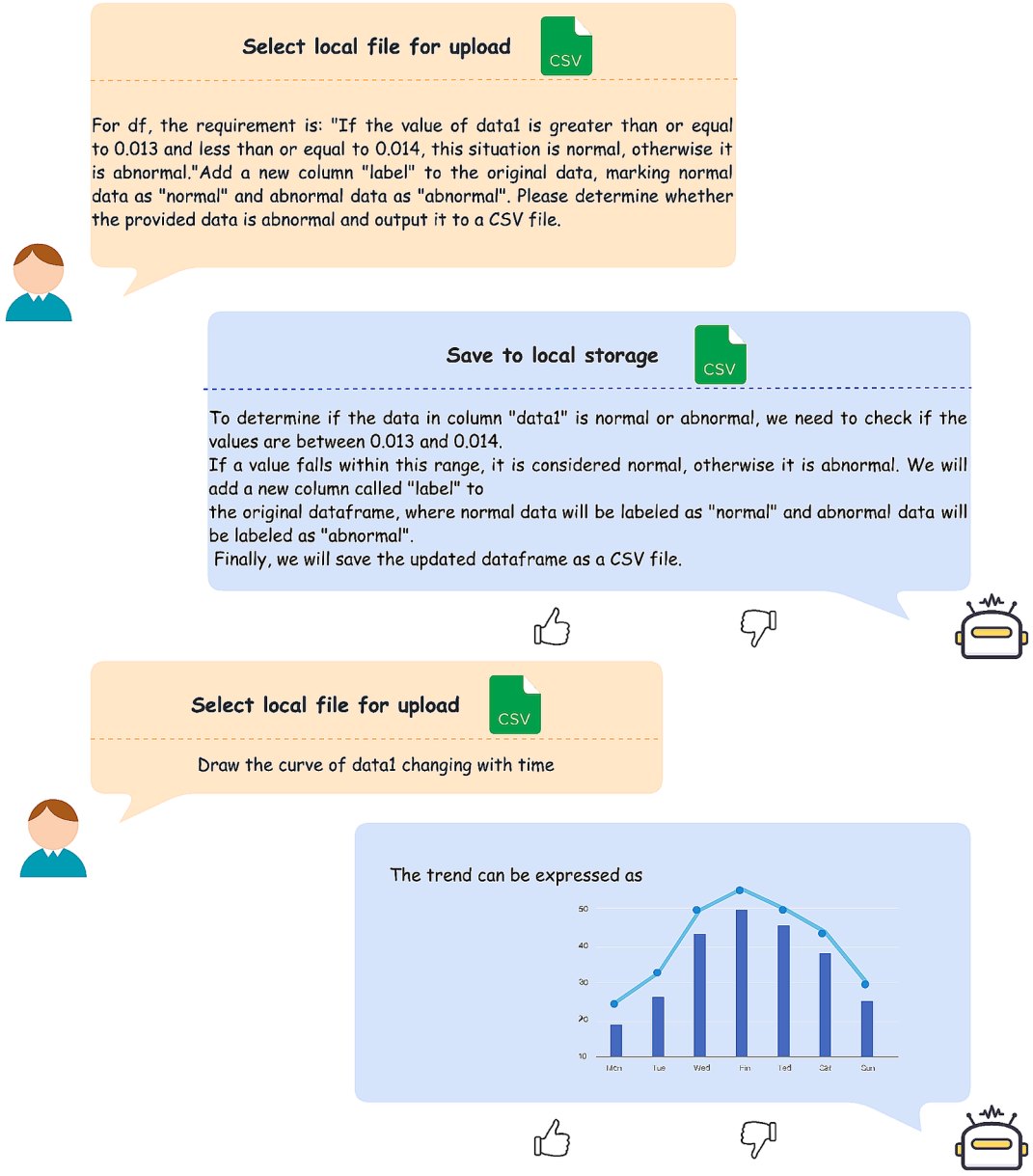


Fig. 11. User dialogue process for data processing operations using LLM: Once the user inputs their question and uploads files, LLM will generate a corresponding response along with a processed CSV file or chart.

depicts how the LLM recommends algorithms based on user input requirements. In contrast, Fig. 14(b)'s fault line graph does not provide readable information for users. Fig. 15(b) demonstrates how the LLM enhances readability by combining prevailing scenarios with domain expert knowledge to interpret warning results.

This paper focuses on readability, autonomy, and generalization to compare the fault diagnosis methods from FD1.0 or FD1.5, which do not use LLM, with those methods using LLM in FD2.0, with results presented in Table 3. According to Table 3, "Without LLM" refers to solely using deep learning methods for fault diagnosis, while "LLM-TSFD" refers to the method proposed in this paper that employs the LLM for fault diagnosis. The roles like the Pipeline Manager, Supervisor, and Controller don't require high readability, so this paper doesn't consider their readability for users. However, the Decision Maker needs to inform users about detected faults, their causes, solutions, and measures. When operators rely on deep learning-based fault diagnosis methods without LLM, users lack readability. In contrast, when combined with knowledge bases, LLM can provide understandable fault information. LLM also enhances the method's autonomy, eliminating the need for significant human or material resources to rewrite program code, as it can be task-driven based on the LLM. This generalization allows its application across various scenarios like those of the Pipeline Manager, Supervisor Controller, and Decision Maker discussed in this paper.

5.2.2. Comparison of FD 1.0, FD 1.5 and FD2.0

This paper conducts a comparative analysis of the FD 1.0, FD 1.5, and FD 2.0 methods. The dataset originates from a steel metallurgy enterprise in Shanghai, China. The training and testing data are derived from industrial continuous casting equipment. The study primarily focuses on data from the current of guide rollers and the crystallizer section. The guide roller currents mainly include the upper roller currents of S1 to S16 and the lower roller currents of S8 to S16. The data from the crystallizer section include the inner arc outlet flow of the broad face, the outer arc outlet flow of the wide face, the proper side outlet flow of the narrow face, the left side outlet flow of the narrow face, and the liquid level of the crystallizer. The datasets comprise 85,000 entries, with 80 %

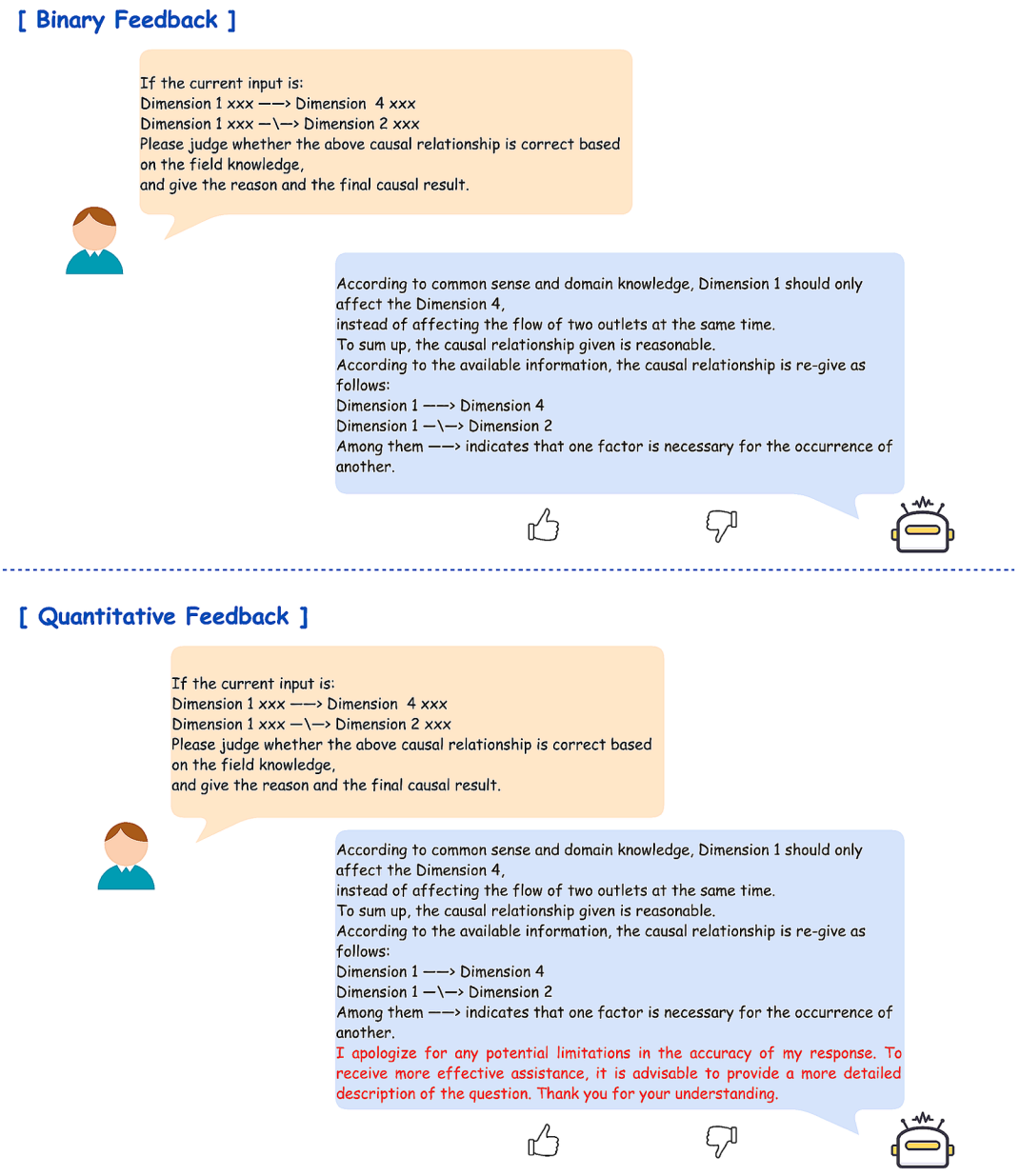


Fig. 12. Process of causality correction operation using LLM: The LLM rectifies the causality of its input by using binary and quantitative feedback mechanisms. The former provides information on whether the answer is correct or incorrect, while the latter informs the LLM that it has provided an inadequate response and requires additional input from the user.

used for the training set and the remaining 20 % for the testing set. The training and testing were conducted on eight A100 GPUs, and the experimental results are presented in Tables 4 and Table 5.

This paper uses Precision, Recall, and F1 Score as evaluation metrics to comprehensively evaluate and compare the performance of the methods employed. Additionally, this paper compares the proposed method with 11 publicly available time series anomaly detection methods: Agglomerative Clustering (Müllner, 2011), Long Short-Term Memory (LSTM) (Karim et al., 2018), Temporal Convolutional Networks (TCN) (Y. He & Zhao, 2019), Mamba (H. He et al., 2024), Transfomer (Wen et al., 2023), iTransformer (Liu et al., 2024), K-means& rules (Budiarto et al., 2019), Autoencoder& expert system (Valtierra-Rodriguez et al., 2023), Knowledge-Distillation (Pol et al., 2023), Time-GPT (Garza et al., 2024), and Prompter-GPT (Alnegheimish et al., 2024). These metrics assess the performance of the methods from different perspectives, providing more comprehensive and precise comparison results.

Precision: Precision measures the proportion of correctly predicted anomalies among all samples predicted as anomalies. It is a statistical metric focused on the data classified as positive by the classifier.

$$Precision = \frac{TP}{TP + FP} \tag{8}$$

Recall: Recall measures the proportion of actual anomalies correctly identified by the model.

$$Recall = \frac{TP}{TP + FN} \tag{9}$$

F1 Score: The F1 Score is the harmonic mean of Precision and Recall, considering the predictions' accuracy and completeness. Its maximum value is 1, and its minimum value is 0, with higher values indicating better model performance.

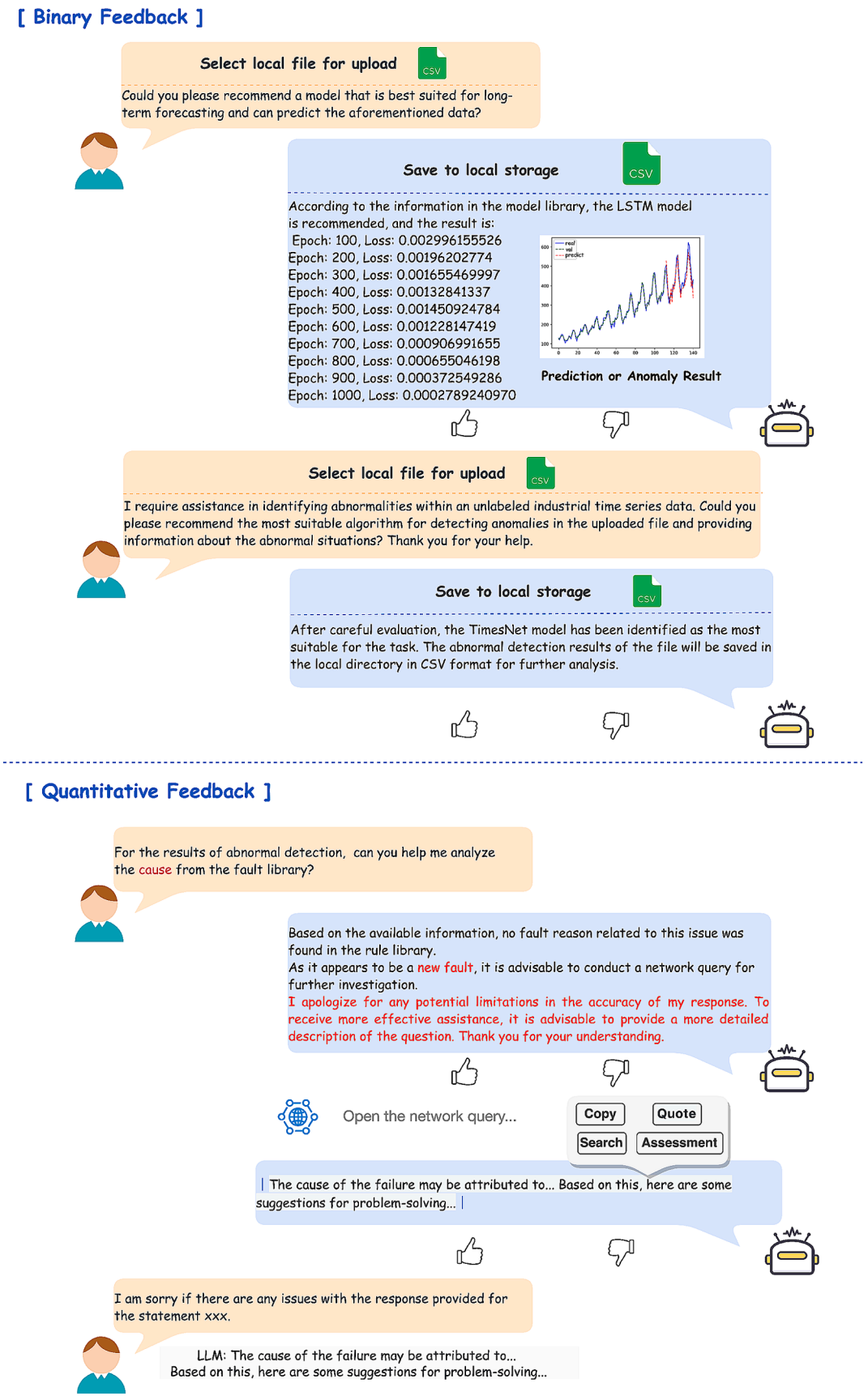


Fig. 13. Fault diagnosis process using LLM: The user inputs a requirement into the LLM, and subsequently, the LLM recommends the most suitable model from its local repository. Furthermore, inquiring about failure causes is also possible through LLM's knowledge base or via an internet search.

(a) Selection of deep learning models (b) Display of anomaly detection/prediction results

Fig. 14. Early warning platform for the steel and metallurgy industry based on deep learning (FD1.0/1.5).

(a) Selection of deep learning models using LLM (b) Display of fault diagnosis results using LLM

Fig. 15. Deep learning-based early warning platform for the steel metallurgy industry (FD2.0).

Table 3
Comparison of anomaly diagnosis results using LLMs versus non-use of LLMs.

Roles	Readability		Autonomy		Generalization	
	Without LLM	LLM-TSFD	Without LLM	LLM-TSFD	Without LLM	LLM-TSFD
Pipeline Manager	–	–	×	√	×	√
Supervisor	–	–	×	√	×	√
Controller	–	–	×	√	×	√
Decision Maker	×	√	×	√	×	√

– means that this article does not consider this situation, × means that it does not have this ability, and √ means that it has this ability.

$$F1Score = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{10}$$

Here, TP represents the number of correctly classified positive samples, FP represents the number of incorrectly classified negative samples, FN represents the number of incorrectly classified positive samples, and TN represents the number of correctly classified negative samples.

Table 4 compares the anomaly detection results of the proposed method with several typical models from the FD1.0, FD1.5, and FD2.0 periods of fault diagnosis. The model with the best anomaly detection results is highlighted in bold at each stage. In the FD1.0 stage, the optimal model is TCN; in the FD1.5 stage, the best performance model is K-means & rules; and in the FD2.0 stage, the proposed method performs the best. Additionally, this paper compares the proposed method with the two methods above in terms of Precision, Recall, and F1 Score. The results indicate that our method outperforms better than the other two in anomaly detection using the current values of various guide rollers in continuous casting equipment.

Table 4
Comparison of FD1.0-FD1.5-FD2.0 anomaly detection results based on the guide roller currents of continuous casting equipment.

Method		Precision	Recall	F1 Score
FD 1.0	Agglomerative Clustering	0.1212	1.0000	0.2162
	LSTM	0.7463	0.7378	0.7420
	TCN	**0.8563**	**0.8465**	**0.8514**
	Mamba	0.6677	0.8250	0.7381
	Transfomer	0.5224	0.6884	0.5940
	iTransformer	0.6798	0.6720	0.6759
FD 1.5	**K-means & rules**	**0.6570**	**0.5497**	**0.5986**
	Autoencoder & expert system	0.2567	0.8408	0.3933
	Knowledge-Distillation	0.5194	0.6418	0.5741
FD 2.0	Time-GPT	0.6237	0.7707	0.6895
	Prompter-GPT	0.8363	0.8463	0.8413
	Ours	**0.9286**	**0.9193**	**0.9239**

Table 5 primarily analyzes the data from the crystallizer section of continuous casting equipment and evaluates several models from the FD1.0, FD1.5, and FD2.0 stages. This paper uses Precision, Recall, and F1 Score as comparison metrics. The models Transformer, Knowledge-Distillation, and the proposed method are highlighted in bold to indicate better performance. Compared to the other two methods, the

Table 5
Comparison of FD1.0-FD1.5-FD2.0 anomaly detection results based on the crystallizer section of continuous casting equipment.

Method		Precision	Recall	F1 Score
FD 1.0	Agglomerative Clustering	0.0305	1.0000	0.0592
	LSTM	0.7078	0.7345	0.7209
	TCN	0.5757	0.7232	0.6411
	Mamba	0.6128	0.7928	0.6913
	Transfomer	**0.7860**	**0.8437**	**0.8138**
	iTransformer	0.7449	0.8305	0.7854
FD 1.5	K-means & rules	0.0847	0.5131	0.1454
	Autoencoder & expert system	0.5971	0.7022	0.6454
	Knowledge-Distillation	**0.7692**	**0.8098**	**0.7890**
FD 2.0	Time-GPT	0.7164	0.7232	0.7198
	Prompter-GPT	0.7706	0.8098	0.7897
	Ours	**0.8111**	**0.8569**	**0.8334**

proposed method detects positive samples with higher precision while maintaining a high recall rate. The F1 Score is also the highest, demonstrating the superior detection performance of the proposed method.

5.2.3. Ablation experiment

This paper compares the average F1 Scores of 11 publicly available models on two datasets, as shown in Fig. 16. The scatter plot displays each model's average F1 Score, with darker colors indicating higher values. The Prompter-GPT model has the highest average score, making as the baseline model for ablation experiments. Additionally, when using a large language model as the Decision Maker for fault detection, we provide an explanatory textual analysis of the results, which differs from the evaluation methods of the Pipeline Manager, Supervisor, and Controller modules. Therefore, the ablation experiments in this section focus on the performance of the Pipeline Manager, Supervisor, and Controller modules in terms of Precision, Recall, and F1 Score to assess their effectiveness in fault detection based on the data collected from continuous casting equipment.

This paper introduces the Pipeline Manager, Supervisor, and Controller modules into the baseline model and evaluates their effectiveness, as shown in Tables 6 and 7. Table 6 presents the results of the ablation experiments using the guide roller current data from continuous casting equipment. From Table 6, it is evident that adding the Pipeline Manager and Supervisor modules to the baseline model enhances Precision, Recall, and F1 Score, confirming the effectiveness of these modules. The Controller module, which utilizes a model recommended by a large language model, replaces the current baseline model. When the Controller module is introduced, there is a significant improvement compared to the baseline model that only includes the Pipeline Manager and Supervisor modules. Compared to the baseline model, the proposed method shows improvements in Precision, Recall, and F1 Score, demonstrating the effectiveness of the proposed modules.

Table 7 displays the results of the ablation experiments using data from the crystallizer section of the continuous casting equipment. The Precision, Recall, and F1 Score improve when the baseline model incorporates the Pipeline Manager and Supervisor modules. Additionally, the combination of the Controller and Pipeline Manager modules outperforms the baseline model with only the Pipeline Manager module. However, the combination of the Controller and Supervisor modules slightly underperforms compared to the baseline model with only the Supervisor module. Nevertheless, using all three modules together surpasses the performance of the baseline model, the baseline model with the Pipeline Manager module, and the baseline model with the Supervisor module. This demonstrates the effectiveness of the proposed Pipeline Manager, Supervisor, and Controller modules.

5.2.4. Comparison of LLM-TSFD and LLM methods without using prompts

This paper employs Decision Maker to provide interpretable responses to user inquiries, and presents a validation dataset in the field of iron and steel metallurgy, consisting of 500 Q&A pairs constructed from user-generated questions and answers. The dataset includes only text descriptions without any accompanying CSV files or images. It covers various aspects such as data processing, causality modification, model recommendation results, fault cause diagnosis results, and related opinions. To evaluate the effectiveness of our proposed approach using the prompt (LLM-TSFD), we compare it with a no-prompt approach using four publicly available LLMs: GLM-3, ERNIE Bot3.5, GPT3.5, and GPT4.0. Four metrics (i.e bleu-4, rouge-1, rouge-2, and rouge-*l*) are used to measure performance and present the results in Table 8.

This paper begins by providing a brief introduction to the four indicators used in this study: The bleu-4 score evaluates the degree of similarity between predicted and standard answers in quaternion cases. Higher values indicate that the predicted results align more closely with the reference text. The rouge-1 metric measures the overlap of words between predicted outcomes and reference answers, specifically

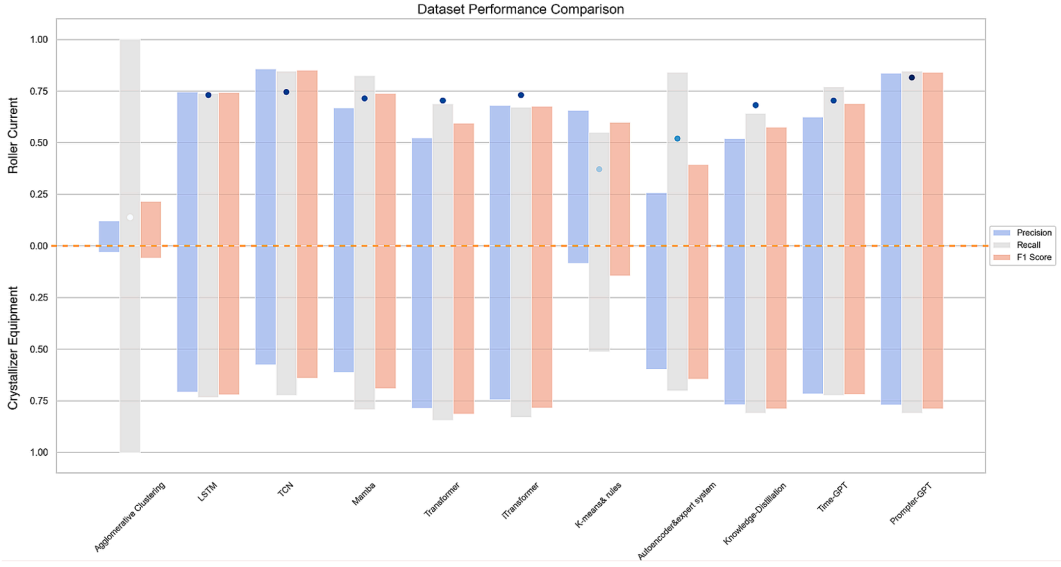


Fig. 16. This paper evaluates the Precision, Recall, and F1 Score of 11 models on different datasets. The gradient-colored circles in the figure represent the average F1 Score of each model calculated across the two datasets. Darker colors indicate higher values. The Prompter-GPT model has the highest average F1 Score, making it as the baseline model for ablation experiments.

Table 6
Ablation experiment comparison based on guide roller current data from continuous casting equipment.

Method				Precision	Recall	F1 Score
Baseline	Pipeline Manager	Supervisor	Controller			
√	–	–	–	0.8363	0.8463	0.8413
√	√	–	–	0.8951	0.8849	0.8900
√	–	√	–	0.8437	0.8464	0.8450
–	√	–	√	0.9124	0.9033	0.9078
–	–	√	√	0.8472	0.8442	0.8457
–	√	√	√	0.9286	0.9193	0.9239

√ indicates modules to consider for inclusion, while – indicates modules not to include.

Table 7
Comparison of ablation experiments based on data from the crystallizer section of continuous casting equipment.

Method				Precision	Recall	F1 Score
Baseline	Pipeline Manager	Supervisor	Controller			
√	–	–	–	0.7706	0.8098	0.7897
√	√	–	–	0.8085	0.8267	0.8175
√	–	√	–	0.7858	0.8315	0.8080
–	√	–	√	0.8100	0.8512	0.8301
–	–	√	√	0.7847	0.8305	0.8070
–	√	√	√	0.8111	0.8569	0.8334

√ indicates modules to consider for inclusion, while – indicates modules not to include.

Table 8
Comparison of results under four different indicators using the proposed prompt (LLM-TSFD) versus not using the proposed prompt (no-prompt).

	GLM-3		ERNIE Bot3.5		GPT3.5		GPT4.0	
	no-prompt	LLM-TSFD	no-prompt	LLM-TSFD	no-prompt	LLM-TSFD	no-prompt	LLM-TSFD
bleu-4	0.00435	0.00910	0.00329	0.00973	0.00433	0.02214	0.11524	0.14925
rouge-1	0.13951	0.30279	0.24042	0.28435	0.17687	0.27980	0.35634	0.40338
rouge-2	0.03576	0.14383	0.04820	0.08530	0.02942	0.03638	0.18857	0.20928
rouge-*l*	0.09553	0.20867	0.16302	0.21147	0.12327	0.18315	0.26200	0.34351

assessing how closely predictions match the reference text's individual words or phrases. Higher metric values suggest a stronger similarity between the results and reference text. The rouge-2 metric quantifies the level of agreement between the predicted outcomes and two consecutive words or phrases in the reference text. A higher score indicates a greater similarity to the manual annotation. The metric known as rouge-*l* quantifies the degree of overlap between predicted results and the longest common subsequence found in the reference text. A higher score on this metric indicates superior performance.

Table 8 illustrates the superior performance of the proposed prompt method over the no-prompt approach across all four publicly available LLMs. To observe the contents of Table 8 more intuitively, a 3-D histogram (Fig. 17) compares the no-prompt method with the LLM-TSFD method. Here, the blue bar signifies the proposed LLM-TSFD method, while the orange bar represents the no-prompt method. Fig. 17 clearly shows that in the rleu-4, rouge-1, rouge-2, and rouge-l indicator regions, the height of the LLM-TSFD bars consistently surpasses those of the no-prompt approach, indicating the superior quality of answers generated by our proposed model.

6. Discussion

The FD1.0 was characterized by a data-driven approach that relied on both manual and automated methods to process time-series data from sensors for human decision-making. The subsequent development of FD1.5 saw the integration of data and knowledge in a more automated manner, enabling the processing of various types of data and enhancing human comprehension of model outputs. With the emergence of LLM, the FD2.0 period is now dominated by the user's task decomposition flow. Models are equipped with autonomous empowerment and self-feedback adjustment capabilities that save substantial resources while handling large-scale complex problems and multimodal data processing tasks. Additionally, LLM enhances human readability of results, thereby improving comprehension ability to some extent.

Despite significant improvements in autonomy compared to traditional machine learning models, the LLM still faces challenges such as high resource environment requirements and susceptibility to hallucinations which require further investigation in future studies. Currently, the proposed method centers on a single data source, specifically CSV files. In real-world scenarios, it is necessary to consider diverse types of data sources. Therefore, future research will explore how to handle multi-modal industrial time-series data. Furthermore, the approach adopted in this study primarily employs publicly available general models. These models lack specific capabilities related to the industrial domain. Consequently, future research would construct an iron and steel metallurgy domain dataset and develop a large language model with distinct industrial domain characteristics. Given the varying levels of expertise among engineers, users exhibit different degrees of understanding of the underlying algorithms. This paper primarily focuses on leveraging LLM in combination with the content of the algorithm library to assist users of different skill levels in selecting appropriate models. Furthermore, considering the inherent reasoning capabilities of LLM, we aim to fully exploit these capabilities for comprehensive model recommendation in future work. This includes recommending updates to model hyperparameters to better adapt to new input variables, among other enhancements. Upon publication of the paper, we will make the pertinent research code available through a GitHub repository.

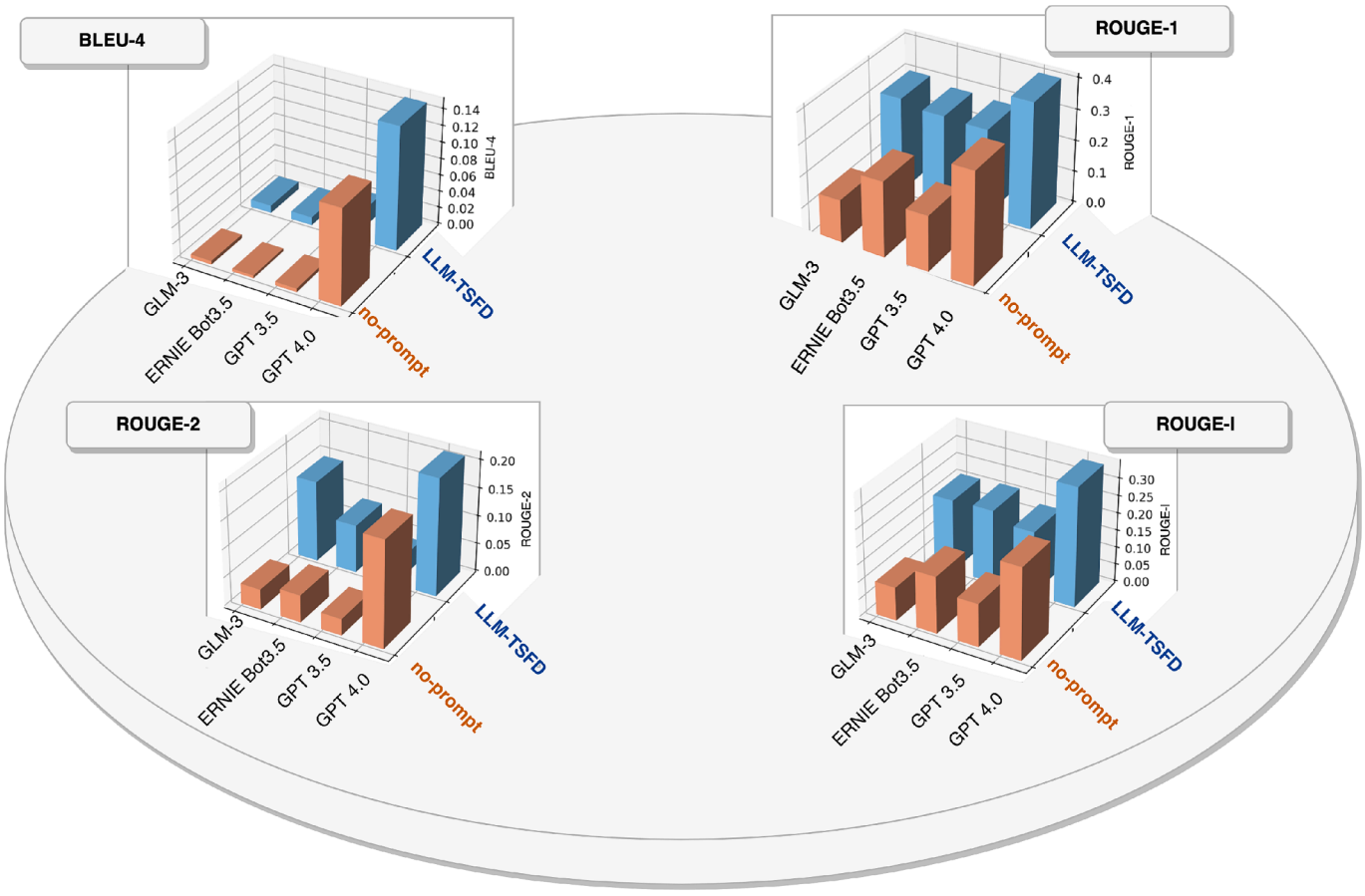


Fig. 17. Comparison of the proposed method with no-prompt methods under four indicators in a 3D bar chart.

7. Conclusions

The emergence of LLM presents a significant opportunity to address the fault diagnosis challenges in iron and steel metallurgy, promoting progress and development in this field. Depending on LLM's data processing and analysis capabilities can obtain accurate prediction and identification of potential faults in the metallurgical process. By analyzing large-scale data, LLM can help engineers identify potential failure points promptly, enabling them to take corrective action before production disruptions or quality issues arise. This combination provides new tools for fault diagnosis that drive greater reliability and efficiency. This paper proposes an LLM-based human-in-the-loop task-driven time-series data fault diagnosis method for application in iron and steel metallurgy. The generative capability of LLM is used to decompose tasks into expected results while feeding back generated results to users for modification through a human–machine feedback collaborative loop. We design a combinatorial reasoning prompt framework applied to industrial time –series fault diagnostic processing capable of adjusting and guiding the behavior and output results of LLM. In iron and steel metallurgy fault diagnosis, we consider four primary roles for LLM: pipeline manager during data processing; supervisor during causal correction; controller during model recommendation; and decision maker during the fault diagnosis stage.

Future research will focus on how best to apply LLM in maintaining data security while protecting industrial privacy. Additionally, using the generative ability of LLM to generate simulation data according to the user's expectations is also one of the key tasks in the future. Moreover, we plan to leverage the reasoning capabilities of LLM to recommend and optimize models for users of varying expertise levels, including hyperparameter adjustments to accommodate evolving input requirements.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Key Research and Development Plan of China under Grant 2019YFB1706300.shusong.

Appendix A

LLM-based fault diagnosis architecture for industrial time series data

The system comprises three functions: chat with data, chat with casual discovery, and chat with diagnosis. Chat with data includes two modules: Data visualization and Data processing. Chat with casual discovery includes Casual discovery visualization and Casual correction modules. Chat with diagnosis consists of Fault diagnosis, Fault prediction, and Fault detection modules. The underlying architecture is implemented using the front-end

framework Dash and CSS, the back-end framework Fastapi, and the algorithm frameworks Pytorch, and TensorFlow. Data interaction is achieved through vector databases and relational databases while integrating Pandas and Sklearn data analysis tools. The LLM backend can be replaced by GPT3.5 or GPT4 as needed. Specific details for each function are shown in the Assets Details section. Finally, Docker is used to package the system for deployment which follows the overall architecture shown in Fig. 18.

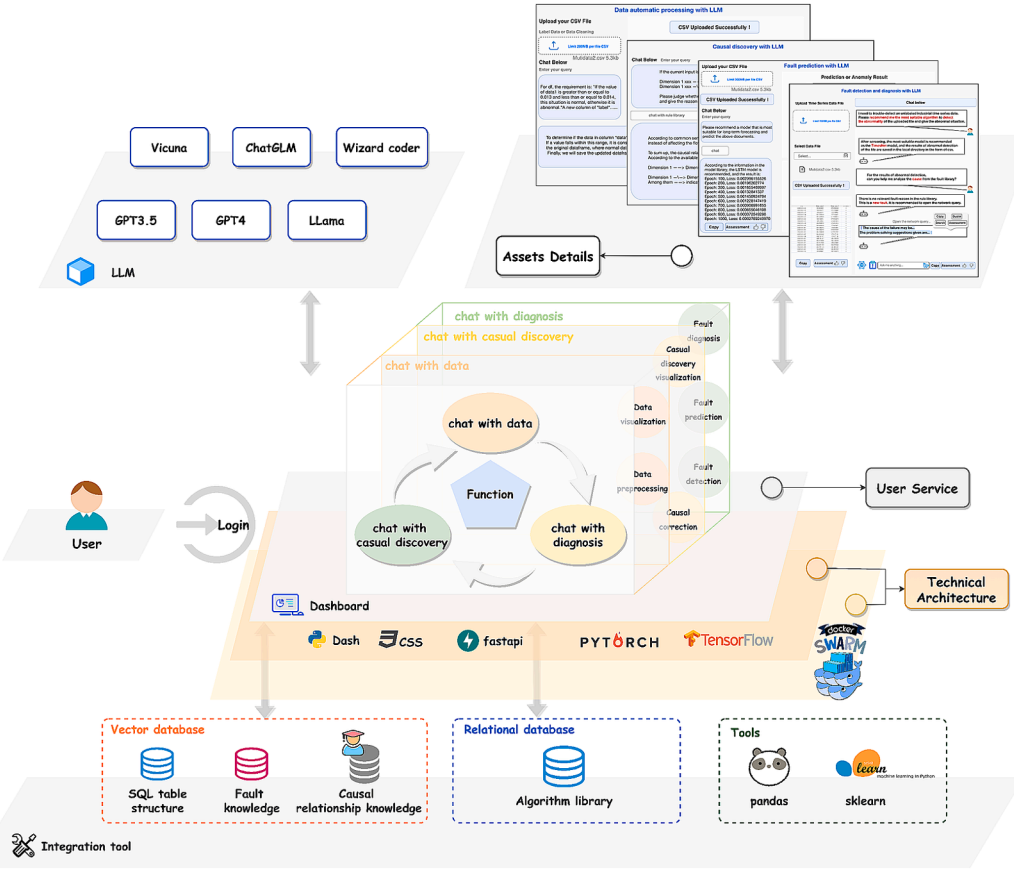


Fig. 18. Architecture of LLM-based fault diagnosis system for industrial time series data.

Chat with data

Data visualization

According to the user's request for data visualization, the system generates and displays the results as shown in Fig. 19. The figure illustrates a trend graph that represents changes in user input data1 over time. Subsequently, a corresponding line graph describing changes over time is generated.

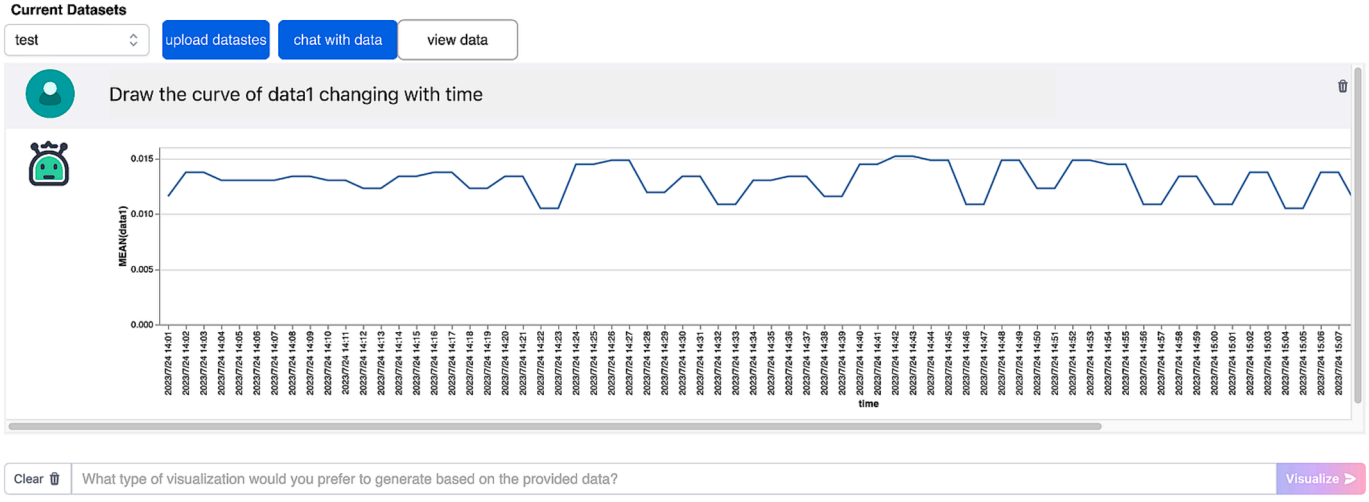


Fig. 19. LLM data visualization results: The grey section represents the problematic records entered by the user, while the line graph depicts the visualization results produced by LLM.

Data processing

This paper focuses on the labeling of simple binary and multi-classified data types, as well as cleaning time series data based on these labels. The original data contains missing values and time irregularities, which requires the use of LLM to complete the labeling and data cleaning operations according to user needs. Table 9 displays sample inputs from users for both binary and multiclassification problems.

Table 9
Sample of user questions in the data analysis module.

Category	Question
Binary Classification Labelling & Data Cleaning	For dimension 1, values between 0.013–0.014 are normal, otherwise they are abnormal. Based on the original data add a "label" column, normal values marked normal, and abnormal values marked abnormal. And simple cleaning of the data, the output as a CSV file.
Multi-Class Classification Labelling & Data Cleaning	For dimension 1, with values between 0.013 and 0.014 labelled as "0.013–0.014", less than 0.013 is labelled as "less than 0.013", and greater than 0.014 is labelled as "greater than 0.014". The data is simply cleaned and output as a CSV file.

After inputting the question into the GPT3.5 model, the output can be classified into two categories, binary classification as shown in Fig. 20, and multiclassification as shown in Fig. 21.

time	quanlity	Dimension 1	data1	Dimension 2	data2	label
2023/4/23 5:01	Good	Dimension 1	0.230769542	Dimension 2	0.682539683	abnormal
2023/4/23 5:02	Good	Dimension 1	0.692307774	Dimension 2	0.682539683	normal
2023/4/23 5:03	Good	Dimension 1	0.692307774	Dimension 2	0.30952381	normal
2023/4/23 5:04	Good	Dimension 1	0.538461767	Dimension 2	0.30952381	normal
2023/4/23 5:05	Good	Dimension 1	0.538461767	Dimension 2	0.416099773	normal
2023/4/23 5:06	Good	Dimension 1	0.538461767	Dimension 2	0.416099773	normal
2023/4/23 5:07	Good	Dimension 1	0.538461767	Dimension 2	0.075963719	normal
2023/4/23 5:08	Good	Dimension 1	0.615384664	Dimension 2	0.075963719	normal
2023/4/23 5:09	Good	Dimension 1	0.615384664	Dimension 2	0.52154195	normal
2023/4/23 5:10	Good	Dimension 1	0.538461767	Dimension 2	0.52154195	normal
2023/4/23 5:11	Good	Dimension 1	0.538461767	Dimension 2	0.524943311	normal

Fig. 20. Data labelling results after LLM processing: binary classification result diagram.

time	quanlity	Dimension1	data1	Dimension2	data2	label
2023/4/23 5:01	Good	Dimension1	0.230769542	Dimension2	0.682539683	less than 0.013
2023/4/23 5:02	Good	Dimension1	0.692307774	Dimension2	0.682539683	0.013-0.014
2023/4/23 5:03	Good	Dimension1	0.692307774	Dimension2	0.30952381	0.013-0.014
2023/4/23 5:04	Good	Dimension1	0.538461767	Dimension2	0.30952381	0.013-0.014
2023/4/23 5:05	Good	Dimension1	0.538461767	Dimension2	0.416099773	0.013-0.014
2023/4/23 5:06	Good	Dimension1	0.538461767	Dimension2	0.416099773	0.013-0.014
2023/4/23 5:07	Good	Dimension1	0.538461767	Dimension2	0.075963719	0.013-0.014
2023/4/23 5:08	Good	Dimension1	0.615384664	Dimension2	0.075963719	0.013-0.014
2023/4/23 5:09	Good	Dimension1	0.615384664	Dimension2	0.52154195	0.013-0.014
2023/4/23 5:10	Good	Dimension1	0.538461767	Dimension2	0.52154195	0.013-0.014
2023/4/23 5:11	Good	Dimension1	0.538461767	Dimension2	0.524943311	0.013-0.014

Fig. 21. Data labelling results after LLM processing: multi-class classification result diagram.

We employed three models, GPT3.5, Vicuna-13b, and Chatglm-6b to evaluate the accuracy of LLM-generated data visualization displays and data processing results. The test involved 80 sets of data to determine whether a valid code was generated or not. Table 10 presents the outcomes obtained with a temperature setting of 0.7 for all three models.

Table 10
Comparison of the accuracy of the results generated by the three models in chat with data.

	Data visualization	Data processing
GPT3.5	0.876	0.578
Vicuna-13b	–	–
ChatGLM-6b	–	–

The code quality generated by ChatGLM-6b and Vicuna-13b in Table 6 is lower, rendering them non-executable. Therefore, they are represented with a hyphen ("-").

Chat with casual discovery

We tested the accuracy of LLM in generating causal relationships using three models: GPT3.5, Vicuna-13b, and Chatglm-6b. The results are presented in Table 11, where we evaluated 80 sets of data to determine whether valid results were generated. All three models had a temperature setting of 0.7.

Table 11
Comparison of the accuracy by the three models of chat with casual discovery.

	Casual Discovery Visualization
GPT3.5	0.634
Vicuna-13b	0.463
ChatGLM-6b	0.327

Chat with diagnosis

To ensure that the fault detection and prediction model is appropriate, it must align with the expected situation. To assess this alignment, we construct a test set for model selection. Sample examples are shown in Table 12. This table demonstrates that if the description contains a majority of specific keywords, then the recommended model on the right-hand side should be considered appropriate; otherwise, it would be an incorrect recommendation. It is important to note that not recommending any model at all also constitutes an incorrect recommendation.

Table 12
Selected samples of model selection test set.

Describe Briefly Contents	Model
a classic linear time series forecasting model, evident trends, seasonal patterns	ARIMA
a classic machine learning algorithm used for fault detection, smaller datasets, non-linear fault detection	KNN
a deep learning-based predictive model, forecasting future, capturing long-term dependencies	LSTM
a classic machine learning, fault detection, linearly separable high-dimensional data	SVM

The information stored in the fault diagnosis database is the information of the fault questionnaires, which is shown in Fig. 22.

Equipment Failure Investigation Form

Unit		Failure Name		Professional Attribute					
Downtime		Management Category		Mainline Work		Important Work		Auxiliary Work	
Fault Process									
Cause Analysis									
Cause Attribute	Production Operation		Insufficient Inspection		Poor Lubrication		Equipment Aging		
	Spare Part Quality		Maintenance Quality		Others				
Countermeasure									
Remarks									

Fig. 22. Information stored in the local knowledge base for fault diagnosis: fault investigation form details.

We tested the accuracy of LLM-generated fault diagnosis results, fault detection recommendation models, and fault prediction recommendation models using three different models: GPT3.5, Vicuna-13b, and Chatglm-6b. The results are presented in Table 13 based on an evaluation of 80 sets of data to determine whether effective results were generated. It is worth noting that all three models had a temperature setting of 0.7 during testing.

your response:

The most suitable model for handling smaller datasets and non-linear fault detection is KNN. The model interface path is path:/Users/zq/Desktop/phd/chatgpt/chat_with_data/pandas-ai-main/LLM_pandas_test/KNN.py

Output from Model:

	time	quanlity	Tagname1	data1	Tagname2	data2	Prediction
70	2023-07-24 15:11:00	Good	big vehicle	0.0134	small vehicle	30.334	0
71	2023-07-24 15:12:00	Good	big vehicle	0.0137	small vehicle	30.3406	0
72	2023-07-24 15:13:00	Good	big vehicle	0.0137	small vehicle	30.3406	0
73	2023-07-24 15:14:00	Good	big vehicle	0.0109	small vehicle	31.3965	1
74	2023-07-24 15:15:00	Good	big vehicle	0.0109	small vehicle	31.3965	1
75	2023-07-24 15:16:00	Good	big vehicle	0.0148	small vehicle	29.7729	0
76	2023-07-24 15:17:00	Good	big vehicle	0.0148	small vehicle	29.7729	0
77	2023-07-24 15:18:00	Good	big vehicle	0.0148	small vehicle	29.0771	0
78	2023-07-24 15:19:00	Good	big vehicle	0.0148	small vehicle	29.0771	0
79	2023-07-24 15:20:00	Good	big vehicle	0.0109	small vehicle	29.7241	0

Fig. 23. Results of LLM recommended fault detection models: relevant time series fault detection models recommended and results generated by running the models.

your response:

The best model for capturing long-term dependencies more effectively is the LSTM model with the interface path of path:/Users/zq/Desktop/phd/chatgpt/chat_with_data/pandas-ai-main/LLM_pandas_test/lstm.py

Image:

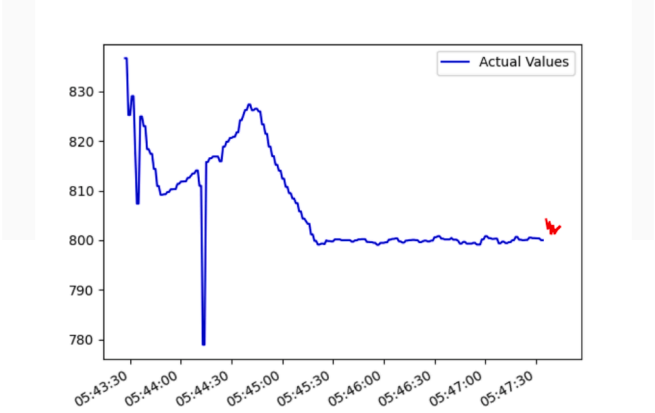


Fig. 24. Results of LLM recommended fault prediction models: relevant time series fault prediction models recommended and results generated by running the models. The red line represents the predicted outcome.

Table 13
Comparison of the accuracy by the three models in chat with diagnosis.

	Recommended Models for Fault Detection	Recommended Models for Fault Prediction	Fault Diagnosis
GPT3.5	0.729	0.649	0.694
Vicuna-13b	–	–	0.450
ChatGLM-6b	–	–	0.389

Data availability

Data will be made available on request.

References

Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., & Rajmohan, S. (2023). Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 1737–1749. https://doi.org/10.1109/ICSE48619.2023.00149.

Alnegheimish, S., Nguyen, L., Berti-Equille, L., & Veeramachaneni, K. (2024). Large language models can be zero-shot anomaly detectors for time series? (arXiv: 2405.14755). arXiv. https://doi.org/10.48550/arXiv.2405.14755.

Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2020). USAD: UnSupervised Anomaly Detection on Multivariate Time Series. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 3395–3404. https://doi.org/10.1145/3394486.3403392.

Bashar, M. A., & Nayak, R. (2020). TAnoGAN: time series anomaly detection with generative adversarial networks. *IEEE Symposium Series on Computational Intelligence (SSCI), 2020*, 1778–1785. https://doi.org/10.1109/SSCI47803.2020.9308512

Budiarto, E. H., Erna Permanasari, A., & Fauziati, S. (2019). Unsupervised Anomaly Detection Using K-Means, Local Outlier Factor and One Class SVM. 2019 5th International Conference on Science and Technology (ICST), 1, 1–5. https://doi.org/10.1109/ICST47872.2019.9166366.

Chai, X. (2020). Diagnosis Method of Thyroid Disease Combining Knowledge Graph and Deep Learning. IEEE Access, 8, 149787–149795. IEEE Access. https://doi.org/10.1109/ACCESS.2020.3016676.

Chauhan, S., & Vig, L. (2015). Anomaly detection in ECG time signals via deep long short-term memory networks. *IEEE International Conference on Data Science and Advanced Analytics (DSAA), 2015*, 1–7. https://doi.org/10.1109/DSAA.2015.7344872

Chen, X., Zhang, N., Xie, X., Deng, S., Yao, Y., Tan, C., Huang, F., Si, L., & Chen, H. (2022). KnowPrompt: knowledge-aware prompt-tuning with synergistic optimization for relation extraction. *Proceedings of the ACM Web Conference, 2022*, 2778–2788. https://doi.org/10.1145/3485447.3511998

Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2023). Automatic Root Cause Analysis via Large Language Models for Cloud Incidents (arXiv:2305.15778). arXiv. http://arxiv.org/abs/2305.15778.

Cheng, L., Li, X., & Bing, L. (2023). Is GPT-4 a Good Data Analyst? (arXiv:2305.15038). arXiv. https://doi.org/10.48550/arXiv.2305.15038.

Du, Z., Qian, Y., Liu, X., Ding, M., Qiu, J., Yang, Z., & Tang, J. (2022). GLM: General Language Model Pretraining with Autoregressive Blank Infilling (arXiv:2103.10360). arXiv. https://doi.org/10.48550/arXiv.2103.10360.

Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2023). PAL: Program-aided Language Models. Proceedings of the 40th International Conference on Machine Learning, 10764–10799. https://proceedings.mlr.press/v202/gao23f.html.

Garza, A., Challu, C., & Mergenthaler-Canseco, M. (2024). TimeGPT-1 (arXiv: 2310.03589). arXiv. https://doi.org/10.48550/arXiv.2310.03589.

Han, S., & Woo, S. S. (2022). Learning Sparse Latent Graph Representations for Anomaly Detection in Multivariate Time Series. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2977–2986. https://doi.org/10.1145/3534678.3539117.

He, H., Bai, Y., Zhang, J., He, Q., Chen, H., Gan, Z., Wang, C., Li, X., Tian, G., & Xie, L. (2024). MambaAD: Exploring State Space Models for Multi-class Unsupervised Anomaly Detection (arXiv:2404.06564). arXiv. https://doi.org/10.48550/arXiv.2404.06564.

He, Y., & Zhao, J. (2019). Temporal convolutional networks for anomaly detection in time series. *Journal of Physics: Conference Series, 1213*(4), Article 042050. https://doi.org/10.1088/1742-6596/1213/4/042050

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2023). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework (arXiv:2308.00352). arXiv. https://doi.org/10.48550/arXiv.2308.00352.

Karim, F., Majumdar, S., Darabi, H., & Chen, S. (2018). LSTM Fully Convolutional Networks for Time Series Classification. IEEE Access, 6, 1662–1669. IEEE Access. https://doi.org/10.1109/ACCESS.2017.2779939.

Kojima, T., Gu, S. (Shane), Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large Language Models are Zero-Shot Reasoners. Advances in Neural Information Processing Systems, 35, 22199–22213.

Li, S., Chen, J., Shen, Y., Chen, Z., Zhang, X., Li, Z., Wang, H., Qian, J., Peng, B., Mao, Y., Chen, W., & Yan, X. (2022). Explanations from Large Language Models Make Small Reasoners Better (arXiv:2210.06726). arXiv. https://doi.org/10.48550/arXiv.2210.06726.

Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., & Long, M. (2024). iTransformer: Inverted Transformers Are Effective for Time Series Forecasting (arXiv:2310.06625). arXiv. https://doi.org/10.48550/arXiv.2310.06625.

Ma, Y., Cao, Y., Hong, Y., & Sun, A. (2023). Large language model is not a good few-shot information extractor, but a good Reranker for hard samples! *Findings of the Association for Computational Linguistics: EMNLP, 2023*, 10572–10601. https://doi.org/10.18653/v1/2023.findings-emnlp.710.

Madotto, A., Lin, Z., Winata, G. I., & Fung, P. (2021). Few-Shot Bot: Prompt-Based Learning for Dialogue Systems (arXiv:2110.08118). arXiv. https://doi.org/10.48550/arXiv.2110.08118.

Müllner, D. (2011). Modern hierarchical, agglomerative clustering algorithms (arXiv: 1109.2378). arXiv. https://doi.org/10.48550/arXiv.1109.2378.

Munir, M., Siddiqui, S. A., Dengel, A., & Ahmed, S. (2019). DeepAnT: A deep learning approach for unsupervised anomaly detection in time series. *IEEE Access, 7*, 1991–2005. https://doi.org/10.1109/ACCESS.2018.2886457

Pol, A. A., Govorkova, E., Gronroos, S., Chernyavskaya, N., Harris, P., Pierini, M., Ojalvo, I., & Elmer, P. (2023). Knowledge Distillation for Anomaly Detection (arXiv: 2310.06047). arXiv. https://doi.org/10.48550/arXiv.2310.06047.

Ruan, J., Chen, Y., Zhang, B., Xu, Z., Bao, T., Du, G., Shi, S., Mao, H., Li, Z., Zeng, X., & Zhao, R. (2023). TPTU: Large Language Model-based AI Agents for Task Planning and Tool Usage. arXiv.Org. https://arxiv.org/abs/2308.03427v3.

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhuang, Y. (2023). HuggingGPT: solving AI tasks with ChatGPT and its friends in hugging face. *Advances in Neural Information Processing Systems, 36*, 38154–38180.

Shridhar, K., Stolfo, A., & Sachan, M. (2023). Distilling Reasoning Capabilities into Smaller Language Models (arXiv:2212.00193). arXiv. https://doi.org/10.48550/arXiv.2212.00193.

Sun, C., Li, H., Li, Y., & Hong, S. (2024). TEST: Text Prototype Aligned Embedding to Activate LLM's Ability for Time Series (arXiv:2308.08241). arXiv. https://doi.org/10.48550/arXiv.2308.08241.

Tu, X., Zou, J., Su, W. J., & Zhang, L. (2023). What Should Data Science Education Do with Large Language Models? (arXiv:2307.02792). arXiv. https://doi.org/10.48550/arXiv.2307.02792.

Valtierra-Rodriguez, M., Rivera-Guillen, J. R., De Santiago-Perez, J. J., Perez-Soto, G. I., & Amezquita-Sanchez, J. P. (2023). Expert system based on autoencoders for detection of broken rotor bars in induction motors employing start-up and steady-state regimes. Article 2 *Machines, 11*(2). https://doi.org/10.3390/machines11020156.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models (arXiv:2203.11171). arXiv. https://doi.org/10.48550/arXiv.2203.11171.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35*, 24824–24837.

Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., & Sun, L. (2023). Transformers in Time Series: A Survey (arXiv:2202.07125). arXiv. https://doi.org/10.48550/arXiv.2202.07125.

Wu, Y., Li, Z., Zhang, J. M., Papadakis, M., Harman, M., & Liu, Y. (2023). Large Language Models in Fault Localisation (arXiv:2308.15276). arXiv. https://doi.org/10.48550/arXiv.2308.15276.

Xin, J. I., Tong-xin, W. U., Zhi-wei, Y., Ting, Y. U., Jun-ting, L. I., & Yu-de, H. E. (2022). Power equipment defect prediction based on temporary knowledge graph. *Journal of Beijing University of Aeronautics and Astronautics, 48*. https://doi.org/10.13700/j.bh.1001-5965.2022.0801.

Yang, H., Yue, S., & He, Y. (2023). Auto-GPT for Online Decision Making: Benchmarks and Additional Opinions (arXiv:2306.02224). arXiv. https://doi.org/10.48550/arXiv.2306.02224.

Yao, Q. (2023). Some Thoughts on the Ecological Construction of Large Models. China Finance.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models (arXiv:2210.03629). arXiv. https://doi.org/10.48550/arXiv.2210.03629.

Zhao, H., Wang, Y., Duan, J., Huang, C., Cao, D., Tong, Y., Xu, B., Bai, J., Tong, J., & Zhang, Q. (2020). Multivariate time-series anomaly detection via graph attention network. *IEEE International Conference on Data Mining (ICDM), 2020*, 841–850. https://doi.org/10.1109/ICDM50108.2020.00093

Zhou, B., Liu, S., Hooi, B., Cheng, X., & Ye, J. (2019). BeatGAN: Anomalous Rhythm Detection using Adversarially Generated Time Series. Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, 4433–4439. https://doi.org/10.24963/ijcai.2019/616.

Zhou, X., Li, G., & Liu, Z. (2023). LLM As DBA (arXiv:2308.05481). arXiv. https://doi.org/10.48550/arXiv.2308.05481.